Project Documentation

C64 Mechanical Keyboard & Kernal Switcher

Project number: 223

Revision: 2

Date: 12.06.2026

C64 Mechanical Keyboard & Kernal Switcher

Standard Version Rev. 2

Module Description

1 Table Of Contents

1 Introduction 3
2 The Keycaps 3
2.1 General considerations 3
2.2 The 1.5U Key Problem (solved) 4
2.3 The Space Bar, PREV, NEXT 5
2.4 Keycap Profiles 5
3 The Switches 6
4 Stabilizers 7
5 The Circuit 9
5.1 Block Diagram 9
5.2 Keyboard Matrix and Output Connector 10
5.3 The SHIFT LOCK Switch 10
5.4 The Micro Controller Module 11
5.5 The Piezo Buzzer 12
5.6 The Power LED 13
5.7 The GPIO Module 15
5.8 The Kernal Switcher 16
5.9 Ultimate Buttons 20
5.10 The RGB LED Strip 22
5.11 I²C-Bus/OLED display 24
6 Connector Pinouts 26
6.1 J1 - Keyboard Connector 26
6.2 J2 – Kernal Switcher 26
6.3 J3 & J8 – Power LED 26
6.4 J4 – Short Board Jumper 26
6.5 J5 – RGB LED Strip 26
6.6 J6 – Reserved 27
6.7 J7 – I²C Bus 27
7 Jumpers 27
7.1 JP1 – SW67/PREV to GND 27

7.2 JP2 – SW68/NEXT to GND 27
7.3 JP3 – Power LED 27
7.4 J4 – Short Board Jumper 27
8 Programming model 28
8.1 Pro Micro 28
8.2 Keyboard Matrix/PCF8575 28
9 Keycap Making 29
10 3D Printables 29
11 Assembly 30
11.1 The Bill of Material 30
11.2 Assembly of the PCB 31
11.2.1 SMD component 31
11.2.2 The Through Hole Components 32
12 Cable Making 36
12.1 The Ribbon Cable ..... Fehler! Textmarke nicht definiert.
12.2 The Dupont Cable(s) ..... Fehler! Textmarke nicht definiert.
13 Wiring Up the Keyboard 36
13.1 USB Keyboard 36
13.2 Keyboard Matrix 36
13.3 Kernal Switcher 36
13.4 Ultimate Buttons 36
13.5 OLED Display (optional) 36
13.6 The LED Strip cable (optional) 36
14 Series Production 37
15 Product References and Disclaimer 37
16 Revision History 37
16.1 Rev. 0 37
16.2 Rev. 1 37
16.3 Rev. 2 37

1 Introduction

This project is a mechanical keyboard for the **C64** or **Ultimate 64**, based on **Cherry MX-compatible switches**.

Mechanical keyboards are not a cheap alternative to the original keyboards. The switches alone (66 + 2) range from about **€0.25 to over €1 per switch**. The stabilizers for the space bar and Return key cost at least **€8**, while good-quality ones can easily cost over **€20**. A set of blank keycaps costs about **€30**. The PCB is surprisingly inexpensive, at less than **€5 plus shipping and import taxes** when ordered from JLCPCB.

Since it was possible, I added some functions that I like and that make this keyboard different from other mechanical keyboards for the C64:

- ***Reset:*** The keyboard can generate ***RESET*** and ***EXROM RESET*** signals.
- ***Kernal switching:*** The keyboard can switch between multiple Kernals on an EPROM Kernal adapter.
- ***OLED display:*** An I²C OLED display can be connected to the keyboard.
- ***Piezo buzzer:*** *The keyboard includes a piezo buzzer for signaling.*
- ***USB keyboard:*** The keyboard can be operated as a ***USB keyboard for VICE and BMC64.***
- ***RGB lighting:*** The keyboard can produce a light show using a ***WS2812B RGB LED strip.***
- ***Power LED:*** The C64's power LED can be connected to the keyboard on either the left or right side and can be made to blink.
- ***Additional buttons:*** Two additional buttons beside the shorter space bar provide additional control functions.

The software runs on a **Pro Micro** module, which is programmed using the free **Arduino IDE** via a USB cable (USB-C or micro-USB).

The keyboard does not require a metal positioning plate above the keyboard PCB because it uses **5-pin switches**: two electrical pins and three centering pins. A **3D-printed frame** holds the PCB in position and interfaces with the C64 case or, in the case of a C64C, the keyboard brackets.

The PCB is attached to the frame using **16 C 2.2 × 6.5 mm screws**. The resulting assembly is sufficiently rigid to provide a wobble-free typing experience.

2 The Keycaps

2.1 General considerations

New keycaps have always been problematic. At least one crowdfunding campaign has failed. There are several ways to obtain suitable keycaps for this keyboard:

- Use old/original keycaps with an adapter pin, available from Retro Fuzion.
- C64 Ultimate-compatible keycaps from CBMStuff may become available in the future.
- Apply the legends onto available blank keycaps.

The keyboard requires:

- 54 × 1U keycaps + 2 × 1U keycaps for PREV and NEXT
- 8 × 1.5U keycaps
- 1 × 2U keycap (RETURN)

- 1 × 7U space bar

The keycaps need legends to be printed on them, except for the space bar.

For more information about the keycaps, please see the **/Keycaps/** subfolder. It contains documentation on how to make or obtain suitable keycaps.

2.2 The 1.5U Key Problem (solved)

The problem is that **original and modern 1.5U keycaps**—such as those used for the **F-keys, Shift, Control, and Restore**—are different (Figure 1).

Modern keycaps, such as blank keycaps or those from the **C64U**, have their switch stem—the part on the underside of the keycap that connects to the switch plunger—located in the center.

Original keycaps have **two switch-stems**, each located **4.75 mm off-center**.

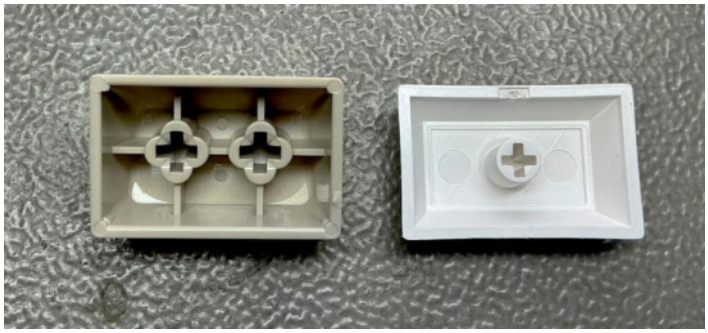


Figure 1: Old versus new 1.5U keycap

The solution is that the switches can be installed in either the **off-center** or **center** position (see Figure 2). To achieve this, the switches are simply rotated by 90°. This does not affect their operation, since the keycap connector is symmetrical.

The **SW**A** position is the **center position**, while **SW**** is the **original off-center position**.

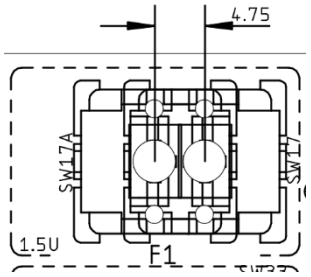


Figure 2: Alternative switch positions for the 1.5U keys

Now it is time to talk about **the U**. It is a unit used to specify key spacing.

Since the early 20th century, typewriter keys have had a spacing of **¾ inch (19.05 mm)**. This did not change with the transition to computer keyboards. The spacing was defined as **19 ± 1 mm** in **DIN 2111** from 1927, **DIN 2139** from 1976, and **ISO 1091** from 1977.

A **1U keycap** fits a key with a 1U spacing. **1.5U keycaps** are used for the **SHIFT, CONTROL, RESTORE**, and **F-key** keys.

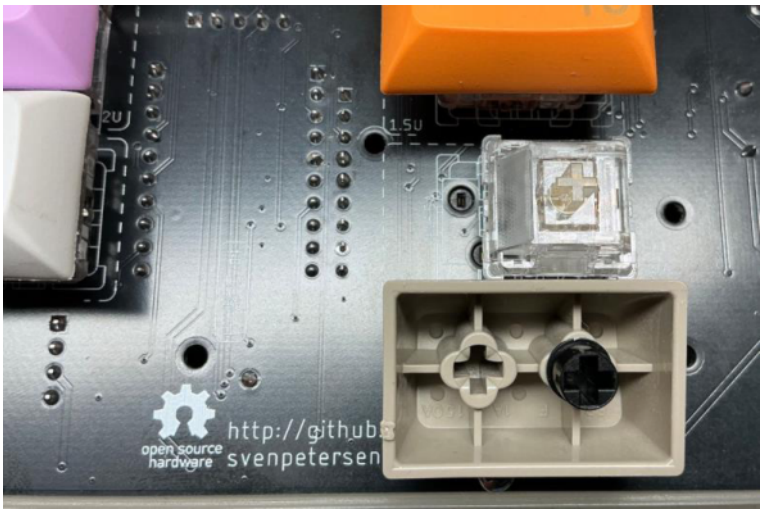


Figure 3: An original F7/F8 keycap with an adapter pin in position. The switch is mounted respectively (PCB Rev. 1 Hot Swap)

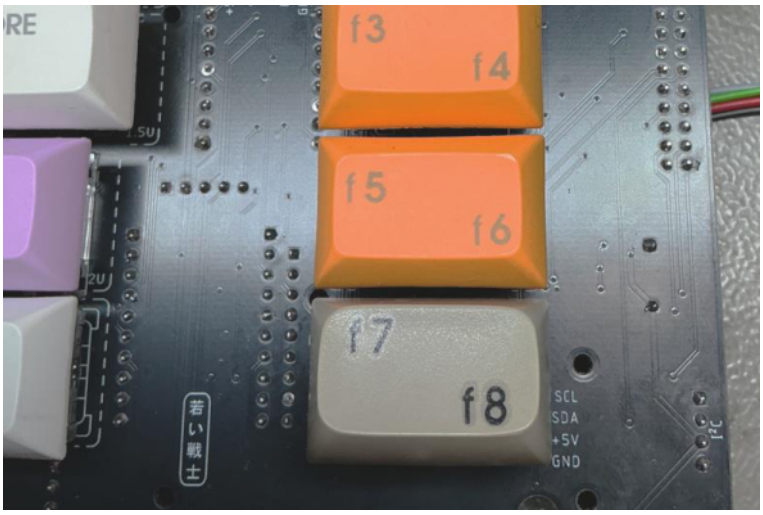


Figure 4: The original keycap mounted on switch

2.3 The Space Bar, PREV, NEXT

The original space bar is **9U** wide, which I could not find available anywhere. I therefore decided to use a **7U space bar**, which is available from several sellers on AliExpress. The remaining **2U** are used for two additional **1U keys**, **PREV** and **NEXT**, which can be used for switching between Kernals or LED modes.

2.4 Keycap Profiles

There are several keycap profiles available (Figure 5). Except for the **CBMStuff** keycaps, all other profile types differ from the original keycaps. Some profiles use a different keycap shape for each row of the keyboard, which would require ten different types of keycaps: four different 1U keycaps, four different 1.5U keycaps, one 2U keycap, and one space bar.

I have only looked for blank keycaps on AliExpress, and all the ones I found were **DSA profile** keycaps. At least this profile offers all the required keycap sizes.

The material of choice is **PBT**. It is well suited for applying legends, including processes involving heat, and it also has a pleasant feel.

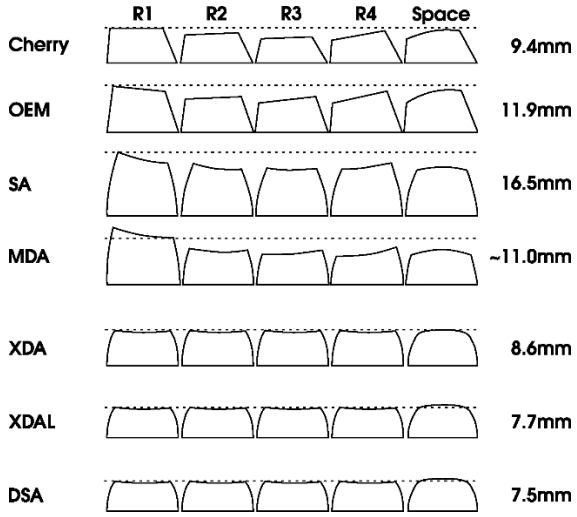


Figure 5: Different keycap profiles

3 The Switches

There are many mechanical switches available, offering different click characteristics and actuation-force profiles. I am not a "keyboard enthusiast," so I do not have any particular preferences in this regard. I have tried switches ranging from budget models to somewhat more expensive ones and found them all quite good—and definitely better than the original keyboard. Even the **€0.25 switches** were perfectly acceptable.

Most importantly, the switches must be **5-pin switches**. All switches have two electrically relevant pins and one plastic pin for centering. Since I did not design a separate mounting plate for this keyboard, the switches rely on two additional plastic pins, making a total of five pins, to position themselves accurately.

A mounting plate is a purely mechanical component made of metal, plastic, or FR4 (the material used for PCBs). It provides a second level, **5 mm above the actual PCB**, with square cutouts for the switch bodies.

I once built a keyboard for 3-pin switches using a second PCB as the mounting plate. Due to the milled cutouts, this was several times more expensive than the keyboard PCB itself. Therefore, **5-pin switches are by far the better choice for this design**.

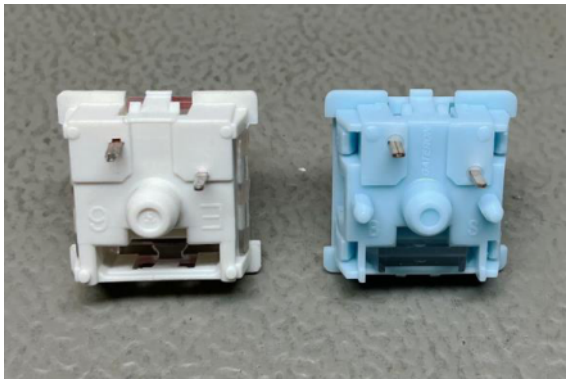


Figure 6: 3 pin versus 5 pin switches (right)

When choosing the switches, avoid **3-pin switches** as well as **magnetic switches**. Magnetic keyboard switches require **Hall-effect sensors**, which are not supported by this PCB.

The **Shift Lock** key does not latch. As far as I know, latching keyboard switches are no longer manufactured. Even if you can still find them, they are prohibitively expensive, costing more than **€30 each**.

4 Stabilizers

The original space bar stabilizer consists of a **spring-steel wire** anchored to the plastic frame and guided through slots in two plastic pins located on the left and right underside of the space bar. It prevents the long keys from tilting or binding when pressed.

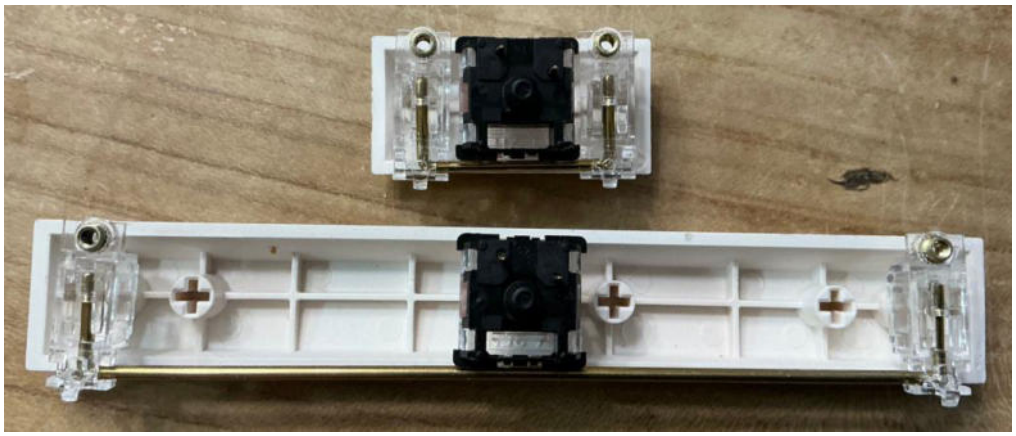


Figure 7: Stabilizer below space bar and RETURN keycap

The **space bar stabilizer** is essential; for the **RETURN** key, it is optional in my opinion.

The stabilizers are the most expensive components of this project. You can expect to pay **€8 to more than €20 for both**. It is important to buy a **7U stabilizer** for the space bar—not a 6.25U stabilizer—and a **2U stabilizer** for the RETURN key. They are often sold as a set.

Screw-in stabilizers are the best choice. There are also stabilizers that snap into the holes in the PCB; both types are suitable. Make sure the stabilizers are designed for a **1.6 mm PCB**, not a 1.2 mm PCB.

Plate-mount stabilizers are not suitable, since this design does not use a mounting plate.

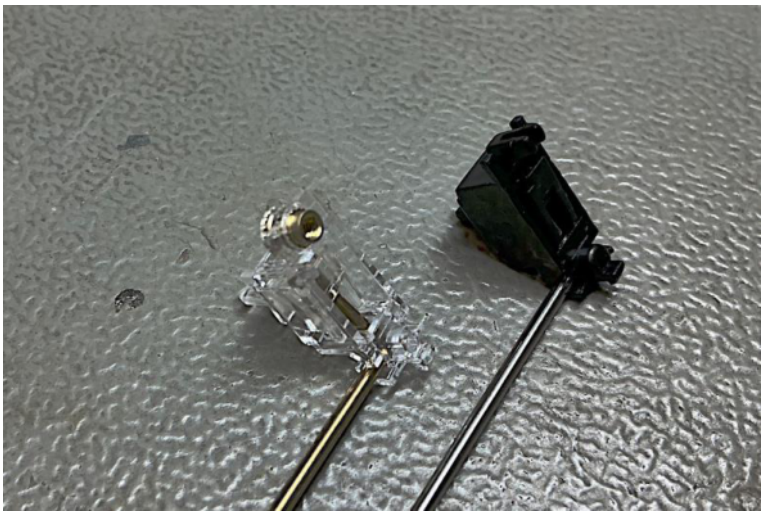


Figure 8: Stabilizer with mounting screw vs snap-in

The stabilizers are mounted in different orientations due to space constraints. The wire runs **above the space bar switch** and **below the RETURN switch** (Figure 9).

Figure 9: Stabilizers mounted on the keyboard

Figure 10 shows a **screw-in stabilizer** mounted on the bottom side of the PCB. One side has a hook that must be hooked into place first. The other side fits through the corresponding mounting hole. Once the stabilizer sits flush against the PCB, it can be secured with the screw. The **plastic insulating washer** should be used.

A 9U stabilizer is only required for the original spacebar. The C64 Ultimate and Blingboard64 spacebar works with a 7U stabilizer.

Find additional information in the **/9U_stabilizer/** subfolder of this repository.

Figure 10: Bottom side of the mounted stabilizer (screw mount)

5 The Circuit

5.1 Block Diagram

This is the block diagram of the C64 keyboard project. The original keyboard consists only of the **switch matrix** (without PREV and NEXT), the interconnections, and the keyboard connector at the end of the cable.

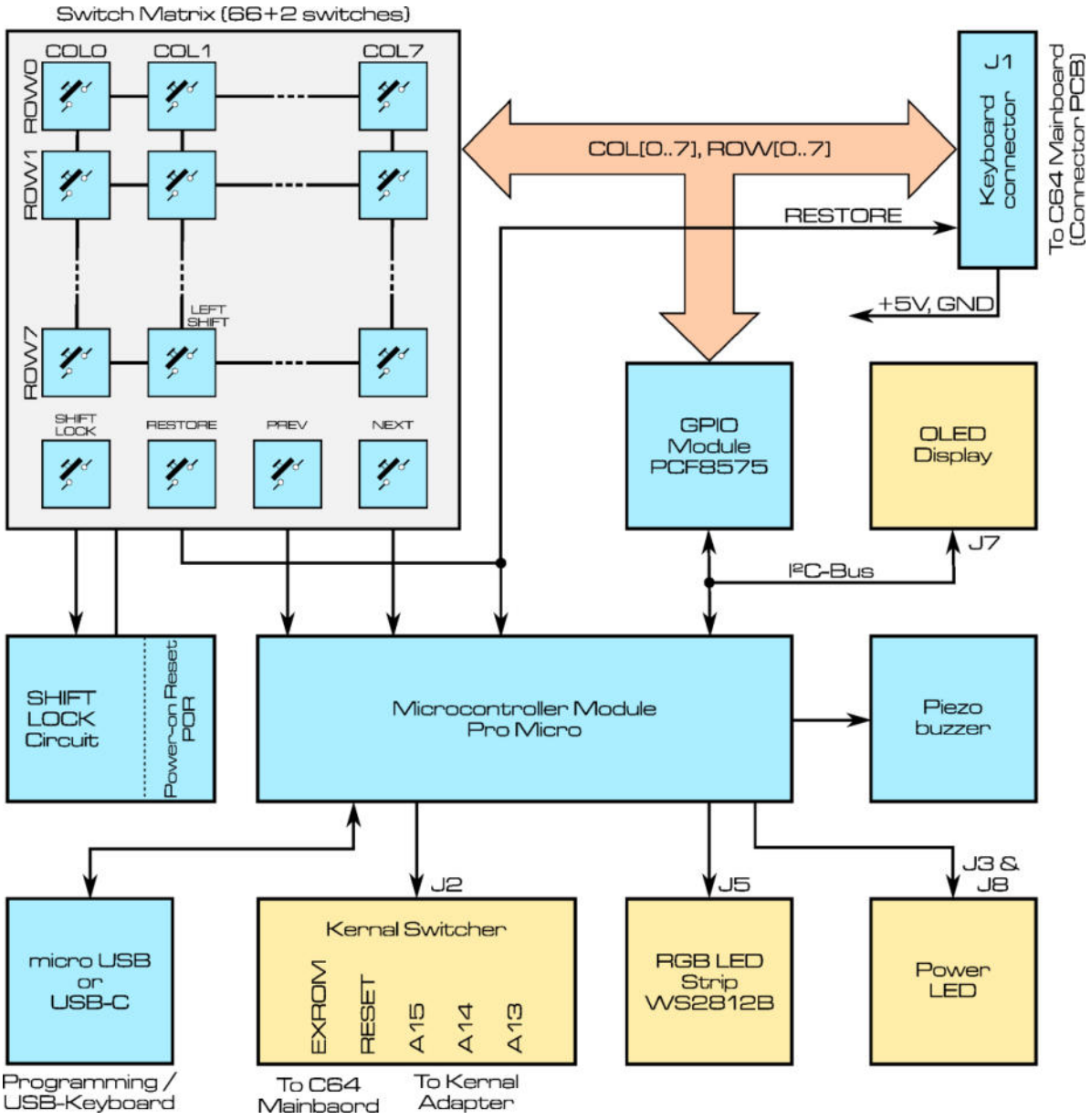


Figure 11: Block Diagram of the C64 Keyboard

The circuit is scalable to suit your needs. The minimum configuration consists of the **switch matrix** (without PREV and NEXT), the **keyboard connector**, and the **Shift Lock circuit**.

If you do not want the beeps, simply leave out the **piezo buzzer**. If you do not need a display, do not connect one; the software must then be configured accordingly.

If you do not want to use the keyboard as a **USB keyboard**, the **GPIO module** is not required.

5.2 Keyboard Matrix and Output Connector

The key switches are connected in an **8 × 8 matrix**, consisting of 8 rows and 8 columns. The **RESTORE** key has a dedicated pin on the keyboard connector. This is identical to the circuit of the original keyboard.

The output connector is a **2 × 10-pin, 90° box header**. I chose this type of connector because it connects to the ribbon cable via an **IDC socket connector**. This requires only a vice to compress the connector and does not require any specialized crimping tools.

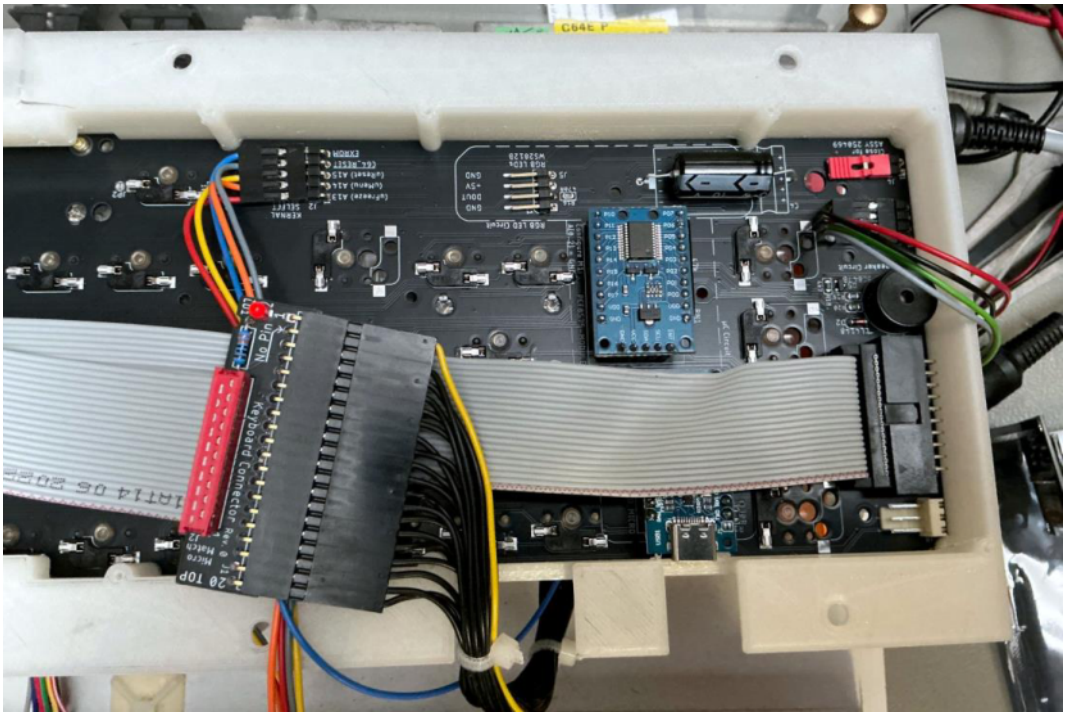


Figure 12: The keyboard connector with the ribbon cable and the connector PCB (Rev 1 with hot swap sockets)

5.3 The SHIFT LOCK Switch

The SHIFT LOCK key does not latch like on an original keyboard. As far as know, latching Cherry MX compatible keyboard switches are no longer being manufactured. Even in case you find a used one of them, they are prohibitively expensive (over 30€ each).

This is the reason for using a standard switch with a little circuit. In Rev. 0, this is not 100% reliable. Sometimes the keyboard starts up with shift lock on. The circuit works perfectly when powering the keyboard from a lab power supply. Only installed in a C64, it sometimes does not work properly. The problem will be resolved in a later revision of the PCB. There is a simple fix with a botch wire, though (see **Fehler! Verweisquelle konnte nicht gefunden werden.**).

The status of the SHIFT LOCK switch is indicated by an LED, which is attached to this switch. It should be a **high brightness** LED, so even in daylight it can be noticed.

The **Power-On Reset (POR)** circuit is only required if the keyboard is intended to be used without the Pro Micro controller, as a **"passive" matrix keyboard**.

An installed Pro Micro resets the Shift Lock circuit. Therefore, if **M2** is installed, the "expensive" POR circuit (IC3) can be left unpopulated.

Figure 13: SHIFT LOCK status LED

I have tried normal LEDs in this position. In a dimly lit room, there was no problem, but in a well-lit room, their visibility was poor.

The **current-limiting resistor for the LED** is **R1**. It is calculated as **100 Ω** for an LED with a 2 V forward voltage drop (red or green) and a forward current (**IF**) of 30 mA.

A standard LED may not be suitable for this relatively high current. A lower current, and therefore a higher resistor value (e.g. 150 Ω), may be more appropriate for normal LEDs.

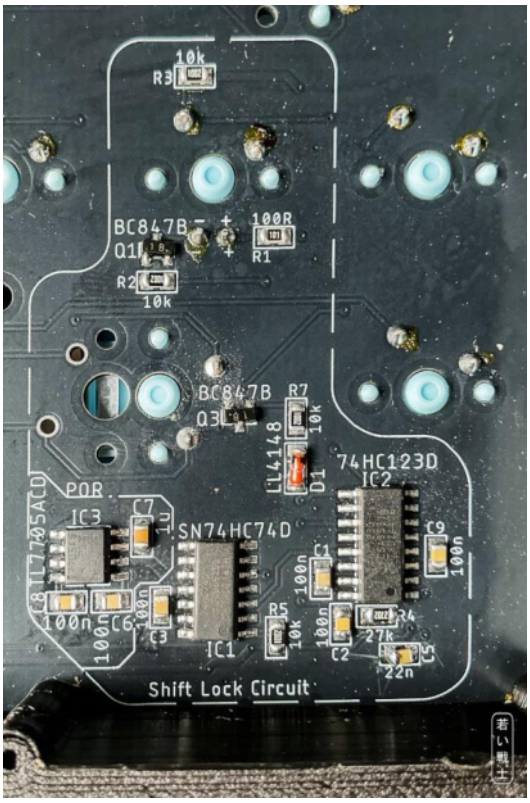


Figure 14: Shift Lock circuit Rev. 1, POR (power-on reset) not populated

5.4 The Micro Controller Module

The keyboard controller is a **Pro Micro**, which is available for about **€5** and can be programmed via a USB cable (micro-USB or USB-C, depending on the model).

I strongly recommend a **Pro Micro with a USB-C connector**, as these are more robust in several respects. It is also preferable to choose a model where the metal connector bracket is secured with

through-hole soldering rather than only with SMD pads. I have already broken off half a dozen micro-USB connectors myself.

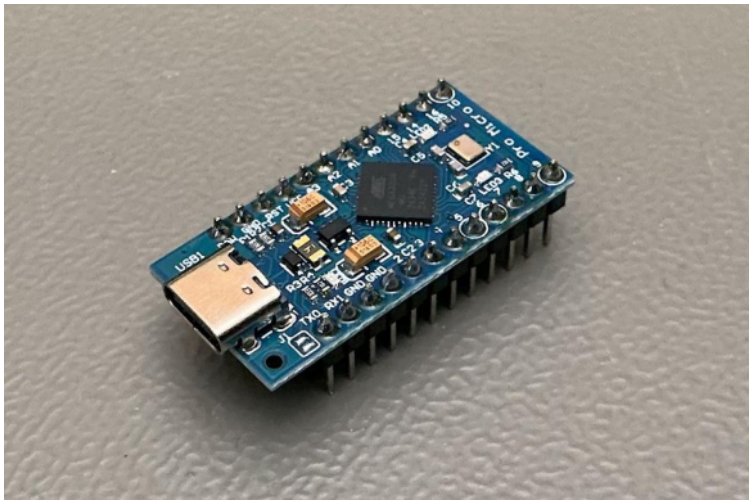


Figure 15: pro micro with a USB-C connector

The Pro Micro is programmed using the **Arduino IDE** via a USB cable. It is configured using `#define` statements at the beginning of the source code. Options that are not required can simply be commented out.

Adding `//` at the beginning of a line turns it into a comment, so configuring the software should be manageable even without a PhD in computer science.

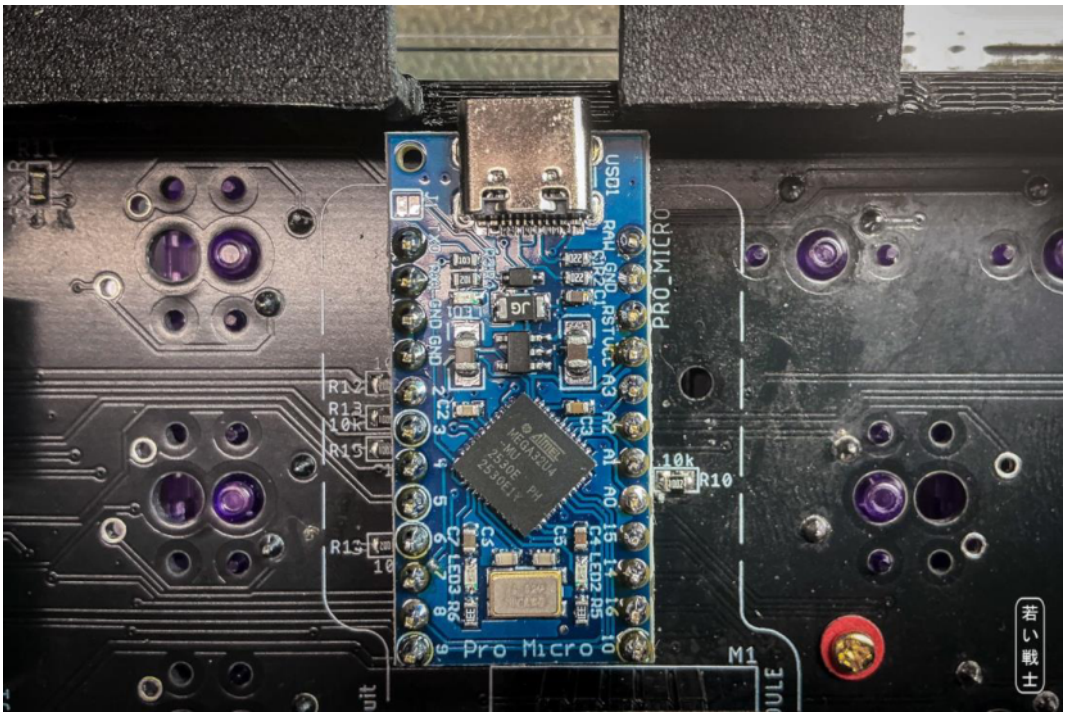


Figure 16:The pro micro on the PCB. The 10k resistors are required.

Without the **Pro Micro**, there is no Kernal switching, OLED display, PREV and NEXT keys, piezo buzzer, or LED strip. That is perfectly acceptable if these additional features are not required.

5.5 The Piezo Buzzer

The **piezo buzzer** is a small speaker that is not exactly Hi-Fi, but it provides useful audible feedback.

It emits a **beep** when RESTORE has been held long enough to trigger a C64 reset, and **two beeps** when it has been held long enough to trigger an EXROM reset. Different beep patterns are used when **PREV** or **NEXT** is pressed.

It can also play the **"Are You Keeping Up…"** jingle during startup if the corresponding `#define` statement is enabled.

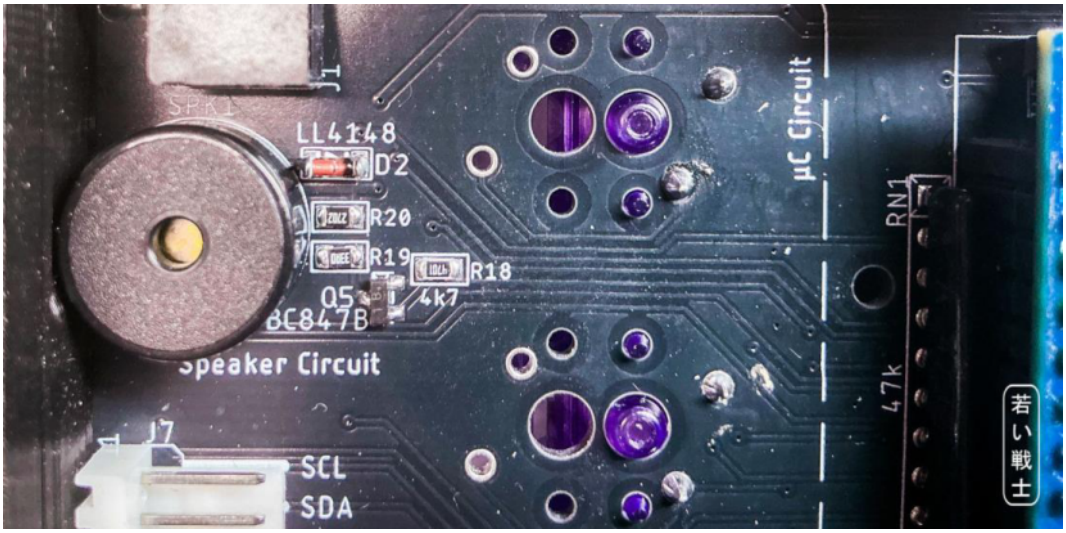


Figure 17: The speaker (piezo buzzer) circuit.

5.6 The Power LED

There are two connectors for the power LED, one on each side of the PCB (**J8 and J3**). This allows you to choose the side that best suits your installation and reduces the number of cables between the top and bottom of the keyboard to the keyboard cable alone.

The power LED is controlled by the **Pro Micro** and blinks in sync with the piezo buzzer beeps. This provides useful visual feedback, especially in noisy environments where the buzzer may be difficult to hear.

Figure 18: The power LED connectorJ8 (top left)

If there is **no controller**, the LED can be powered permanently by closing the **JP3 solder bridge** on the top side of the PCB. Only close JP3 if **no Pro Micro is installed**; otherwise, the controller may be damaged.

The current-limiting resistor is R11, which is 330 Ω by default. This provides a forward current of approximately 10 mA for a standard red LED.

Only one power LED should be used. Using two LEDs may cause them to be dim, have different brightness levels, or fail to light at all.

5.7 The GPIO Module

The purpose of the **GPIO module (M1)** is to provide the Pro Micro with the additional I/O pins required to scan the keyboard matrix **125 times per second** when the keyboard is used as a **USB keyboard for VICE**.

In all other operating modes, the GPIO module is not required. Consequently, the **pull-up resistor network RN1** is not required either.

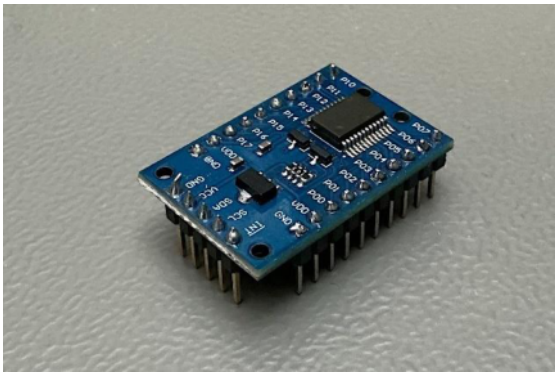


Figure 19: The PCF8575 GPIO module

The module is connected to the pro micro via the I²C bus. To access any IC or module on this bus, a properly set I²C slave address is required to properly access the module. This slave address of the module is configured by solder bridges on the bottom side of the module PCB. They (A0…2) should all be closed to the GND side (Figure 20). That sets the slave address to 20_{HEX}/0x20.

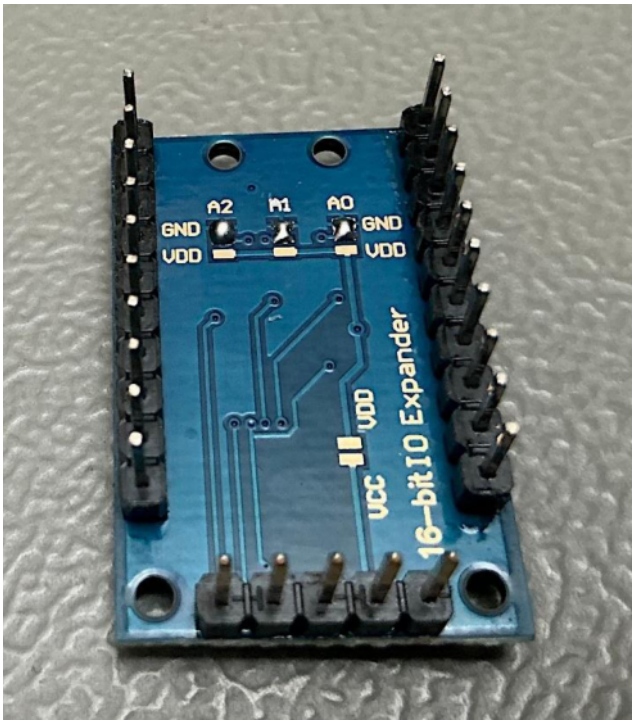


Figure 20: properly configured PCF8575 module

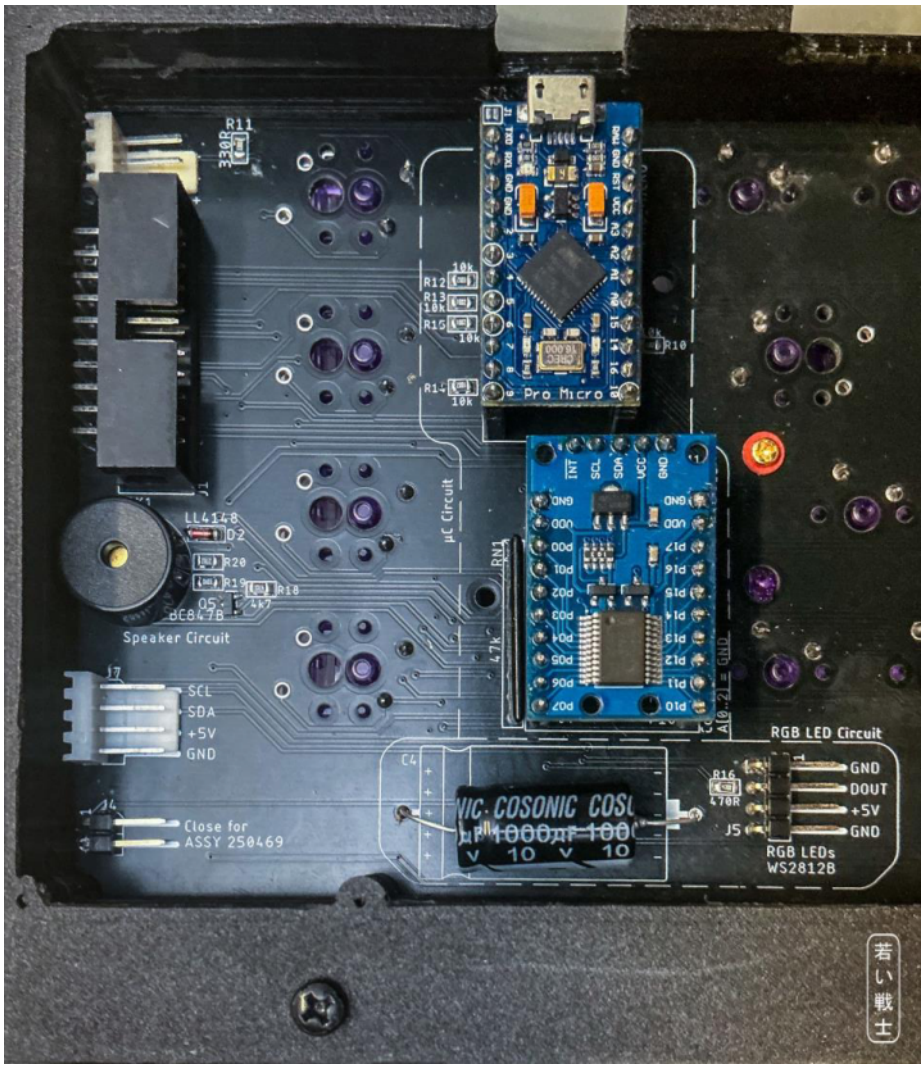


Figure 21: The µC circuit including the PCF8575 GPIO module M1 and resistor network RN1

5.8 The Kernal Switcher

The **Kernal switcher** is controlled by the **RESTORE, PREV, and NEXT** keys. The switching logic is handled by the **Pro Micro** controller.

It can switch between the Kernals stored in an **EPROM** on a Kernal adapter (Figure 22). A **27C512 EPROM** can hold up to **eight Kernals**.

Figure 22: Kernal Adapter

I have created two different versions, both of which are available on **GitHub**:

- C64 Kernal Adapter/Switch Long Board
- C64 Kernal Adapter/Switch Short Board

The reason for the two different versions—for **short boards (ASSY 250469)** and **long boards (all other ASSY numbers)** is that the short boards use a combined **Kernal and BASIC ROM** (16 kB), whereas the long boards have two separate ROMs (8 kB each).

A **27C512 Kernal EPROM** has a capacity of 64 kB and can therefore hold **eight 8 kB Kernals** on a long board. On a short board, the lowest 8 kB memory slot contains the BASIC ROM, leaving the subsequent seven slots for Kernals.

The three highest address bits, **A[13..15]**, are used to select the active Kernal. The Kernal adapters mentioned above have jumpers that normally select the Kernal. However, these address lines can also be controlled by the keyboard. The lower row of pins on the adapter, from left to right, is **A13, A14, A15, and +5 V**. The +5 V connection is not used.

A cable from the keyboard to the Kernal adapter (Figure 23, **yellow/orange/red wires: A15, A14, A13**) already provides the three address signals. A **RESET** connection is also required.

RESET can be tapped at a suitable point near **pin 3 of the user port**. Usually, there is a via nearby that can be used to solder in a single-pin header. This is the **grey wire in the upper part of** Figure 23

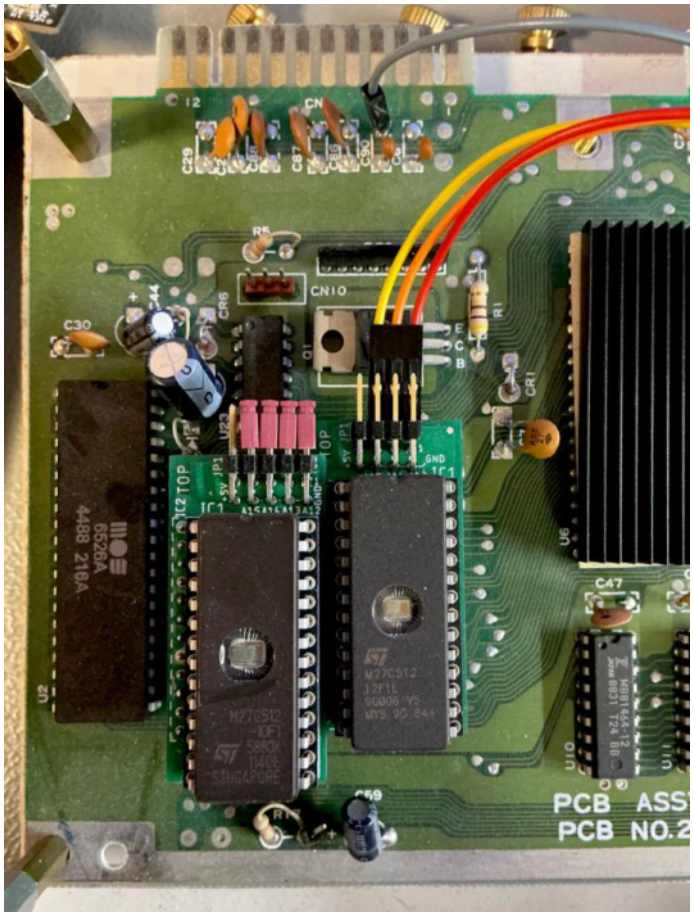


Figure 23: Character set (left) and Kernal adapter (right) on a short board.

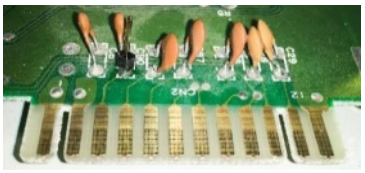


Figure 24: Reset Pin on ASSY250469 (short board)

Figure 25: Reset Pin on ASSY 250425 (long board)

An **EXROM connection** completes the functionality. An EXROM reset can be useful for getting out of **fake cartridge signature (CBM80)** situations used by some games as copy protection.

It is best to tap the **EXROM signal directly at the Expansion Port connector**.

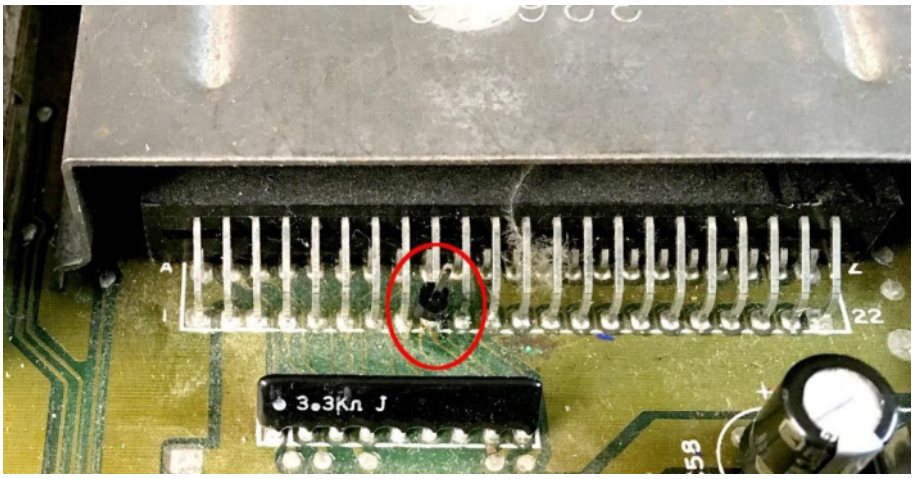


Figure 26: EXROM pin soldered to pin 9 of the expansion port

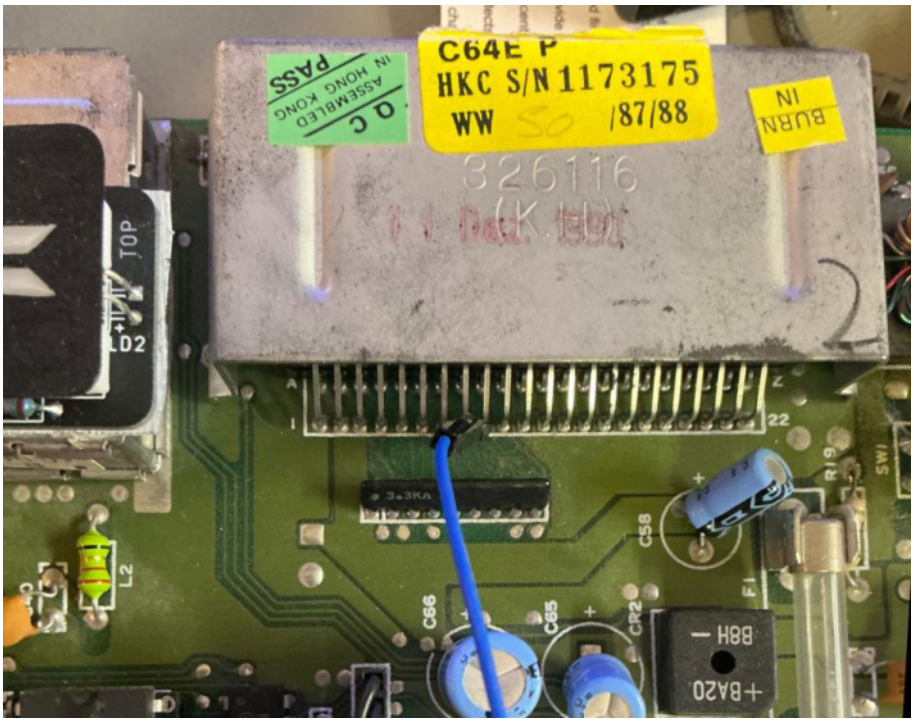


Figure 27: EXROM wire connected to the pin header

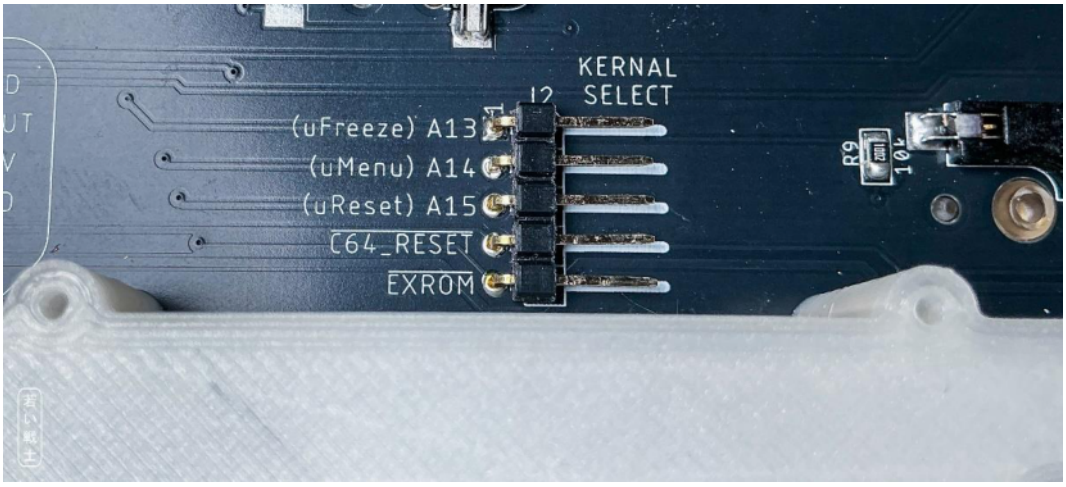


Figure 28: The connector for the Kernal switcher J2

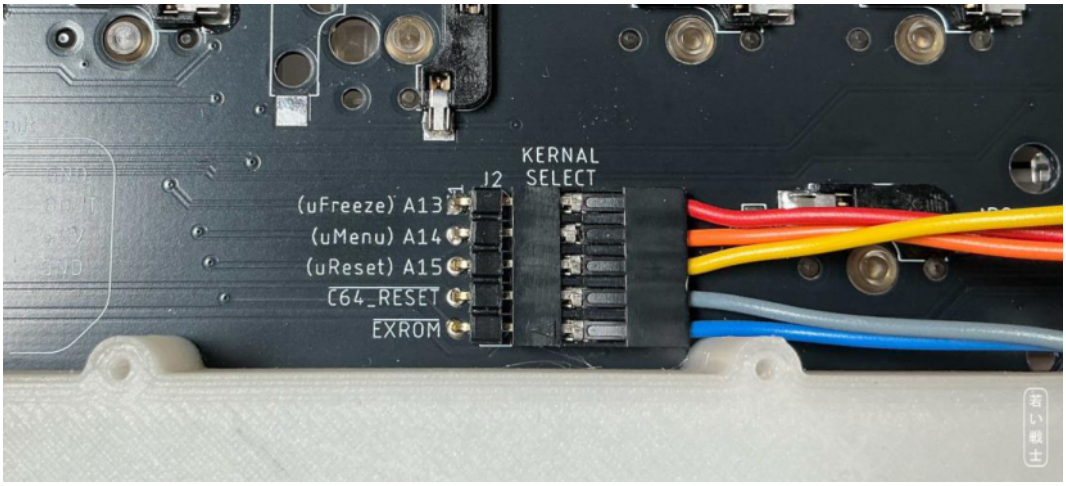


Figure 29: J2 with Kernal Switcher Cable

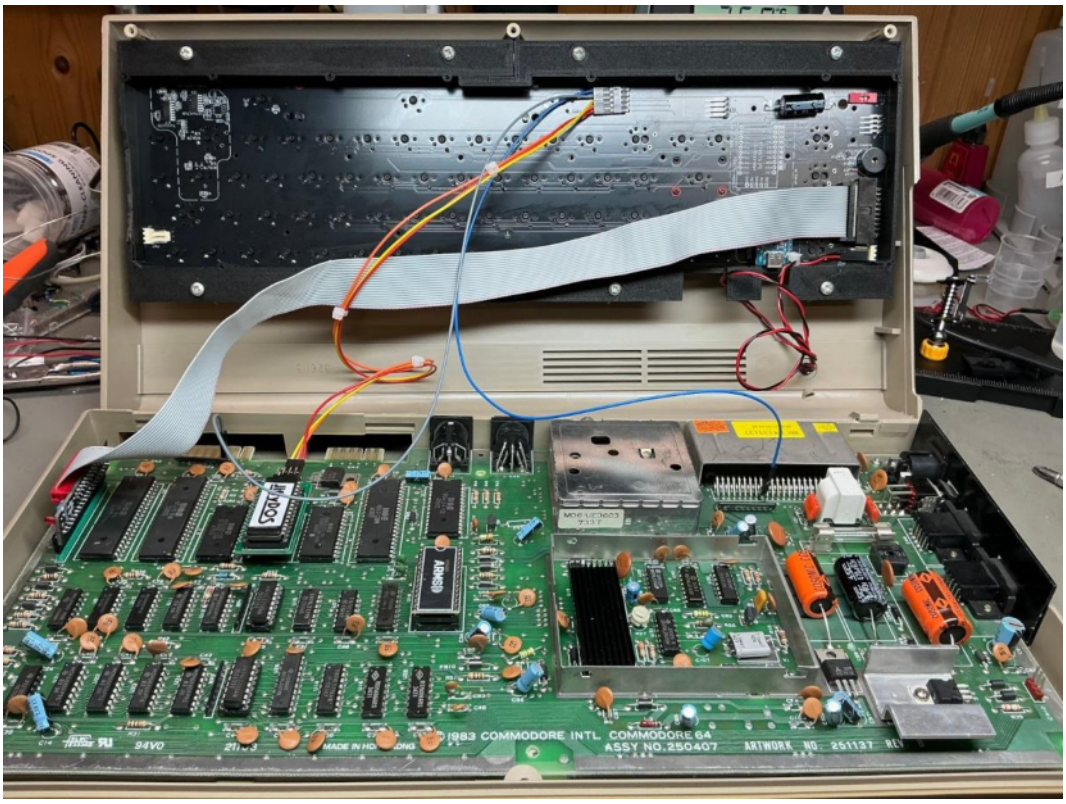


Figure 30: A properly wired-up C64 Breadbin with the installed Kernal cable

5.9 Ultimate Buttons

When connected to the Ultimate 64-Elite -II, the Kernal Switcher is not required. Instead, the buttons pin header on the mainboard is controlled by the keyboard (in case this option is enabled in the Software). Since the outputs are simulated open collector, the C64 keyboard can handle the required 3.3V logic of the Ultimate 64 or the C64 ultimate.

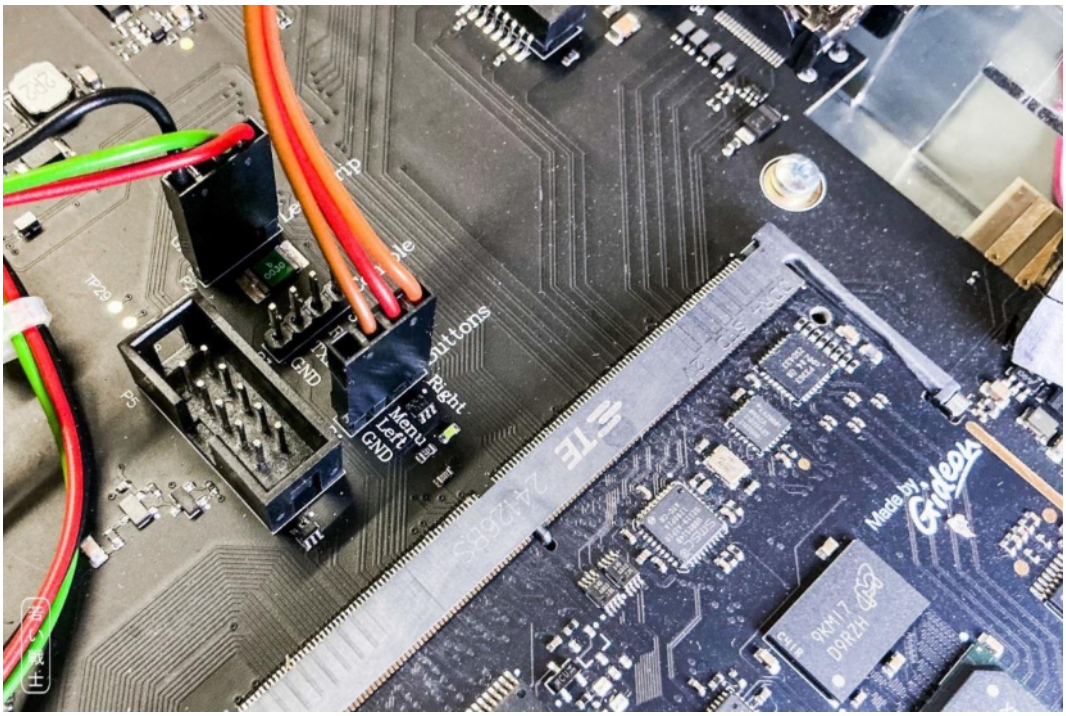


Figure 31: Ultimate elite II buttons pin header

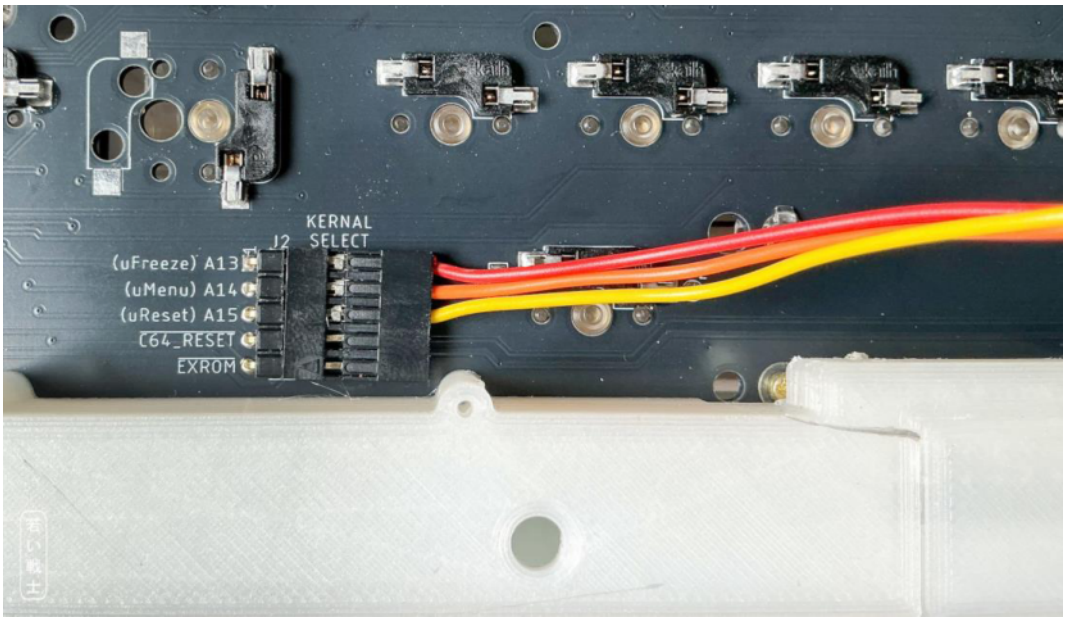


Figure 32: J2 with the Ultimate buttons cable

An important part of the Kernal Switcher is the jumper J4, that selects long board (no jumper placed) or short board (jumper placed).

An important part of the **Kernal switcher** is the **J4 jumper**, which selects the appropriate C64 board type:

- **Long board:** J4 open (no jumper installed)
- **Short board:** J4 closed (jumper installed)

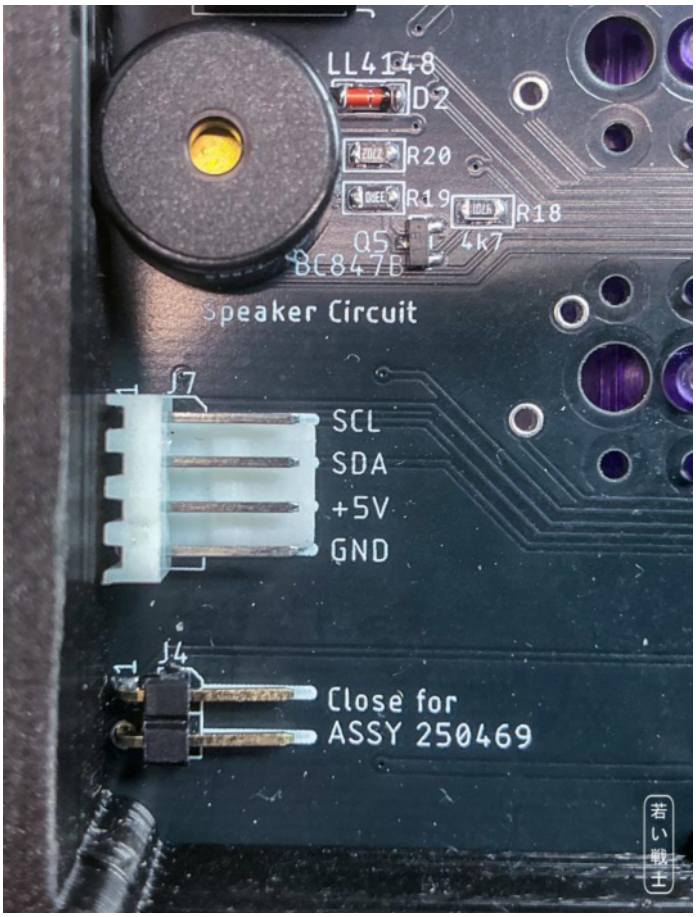


Figure 33: Jumper J4 and the I²C Bus connector J7

The reason for this distinction is that a **27C512 64 kB EPROM** on a long-board adapter can contain up to **eight Kernals**, whereas the first 8 kB of the EPROM on a short-board adapter—at least on the adapter I designed—is reserved for **BASIC**. Therefore, the short-board adapter can contain only **seven Kernals**.

The **J4 jumper** prevents the keyboard from selecting the 8 kB BASIC block as a Kernal. Without this protection, selecting that block would result in a **temporary black screen**.

5.10 The RGB LED Strip

A free pin on the **Pro Micro** is dedicated to controlling a **WS2812B-based RGB LED strip**. The color and brightness of each individual LED can be controlled independently.

The Pro Micro contains **10 different light patterns**, either using a rainbow spectrum (multicolor) or a single color with varying brightness. The patterns are designed to change slowly and smoothly, without distracting or nervous effects.

The length of the LED strip can be defined in the software's source code. It should **not exceed 23 LEDs**.

The computer's **PSU** must be capable of supplying the additional current required by the RGB LED strip. At full brightness (**RGBbright = 255**), the strip draws approximately **400 mA** of additional current. A more moderate brightness setting is **RGBbright = 32**, which requires approximately **50 mA** extra.

The RGB strip consists of **WS2812B LEDs** connected in a daisy chain. Each LED has a **Din (data input)** and **Dout (data output)**. The strip must be connected with the **Din side** to the keyboard's **J5 connector** (Figure 36).

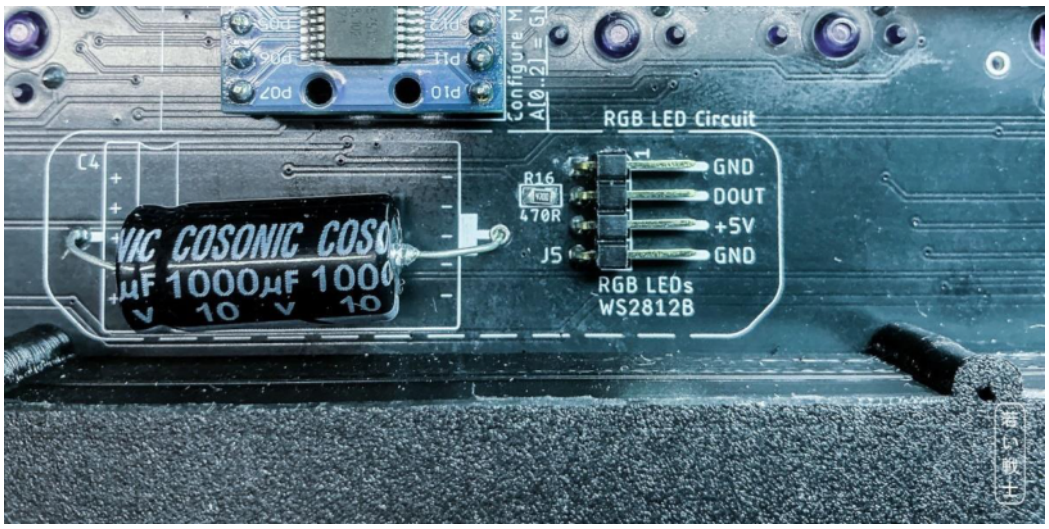


Figure 34: RGB LED Circuit

The **RGB LED circuit** consists of only a **1000 µF electrolytic capacitor**, a 470Ω resistor, and the pin header itself.

If no RGB LED strip is connected, these components are not required.

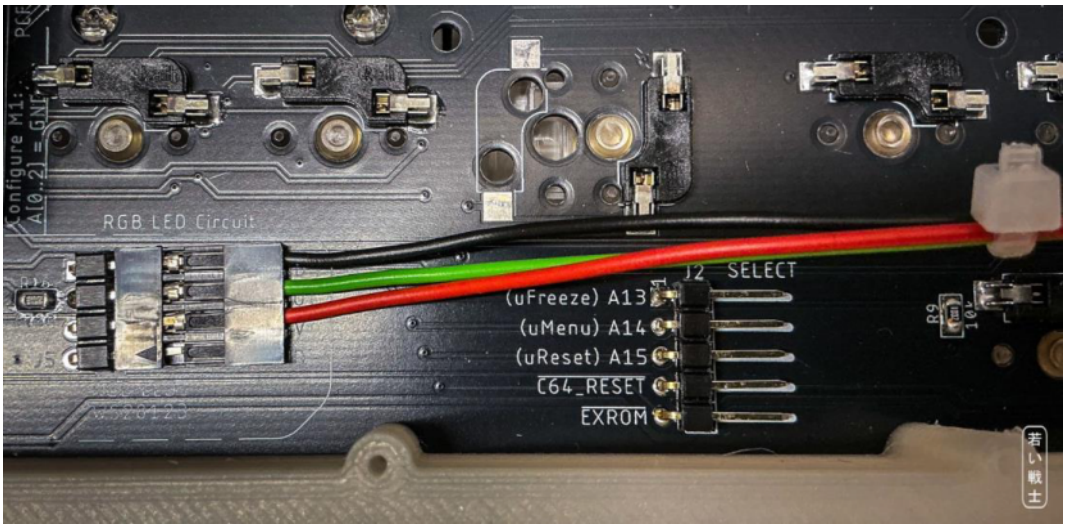


Figure 35: LED Strip Cable at J5 (Board version with hot swap sockets)

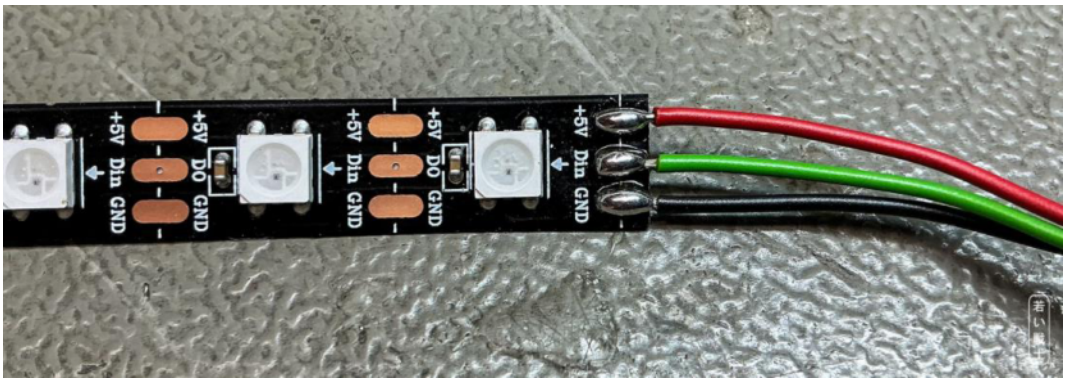


Figure 36: RGB LED Strip with soldered cable

5.11 I²C-Bus/OLED display

The **I²C bus** is a serial bus used to communicate with components, ICs, and modules within electronic devices. It can be used to connect various devices, such as displays and sensors.

I have integrated support for **three different OLED displays** into the Pro Micro software. The desired display can be selected in the source code.

Size	Resolution	Controller
0.91″	128 x 32 pixel	SSD1306
0.96″	128 x 64 pixel	SSD1306
1.3″	128 x 64 pixel	SH1106

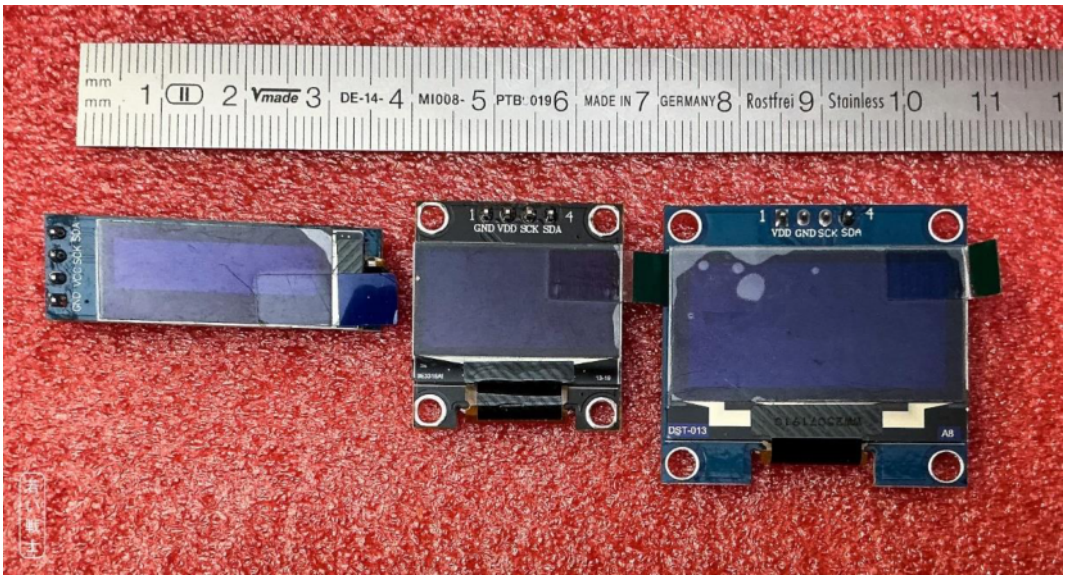


Figure 37: 0.91", 0.96" and 1.3" OLED Display

The display shows the currently selected **Kernal** and **LED pattern** (with software version **v0.00**). During startup, it displays the **C64 logo** and the software version.

When a **RESET** or **EXROM RESET** is triggered, the display indicates which type of reset was performed.

On an **Ultimate 64**, the **FREEZE**, **MENU**, and **RESET** button presses are also displayed.

The cable of the OLED display is a Dupont cable. It might be required to solder the cable to the display, depending on how you want to install it.

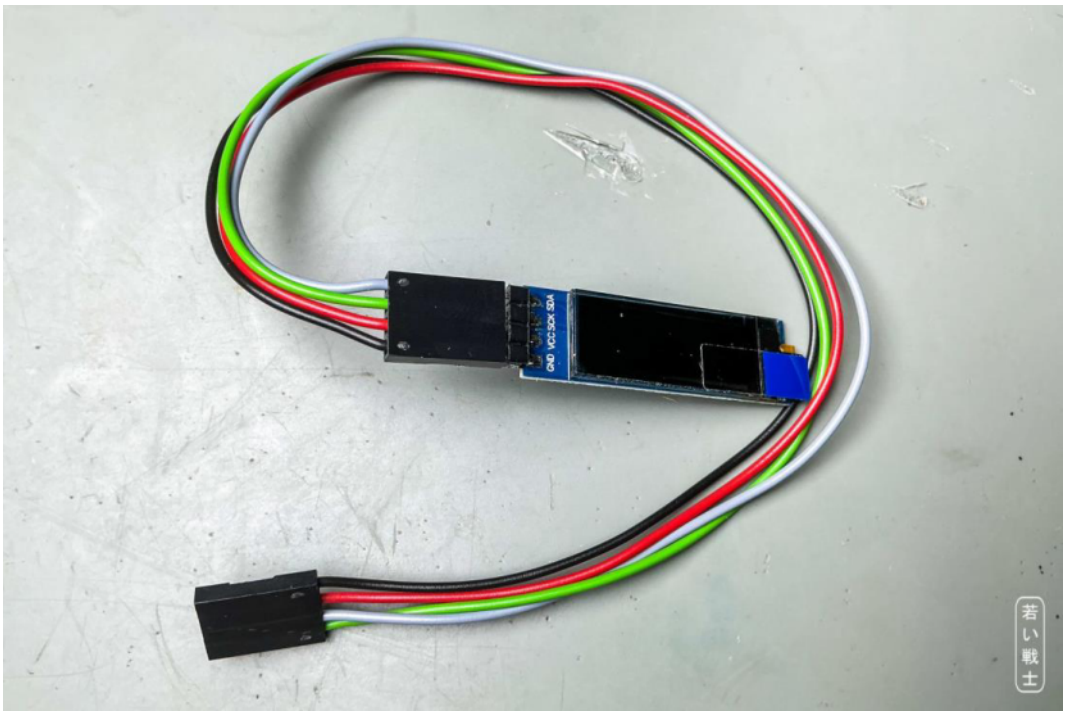


Figure 38: 0.91" OLED display with cable

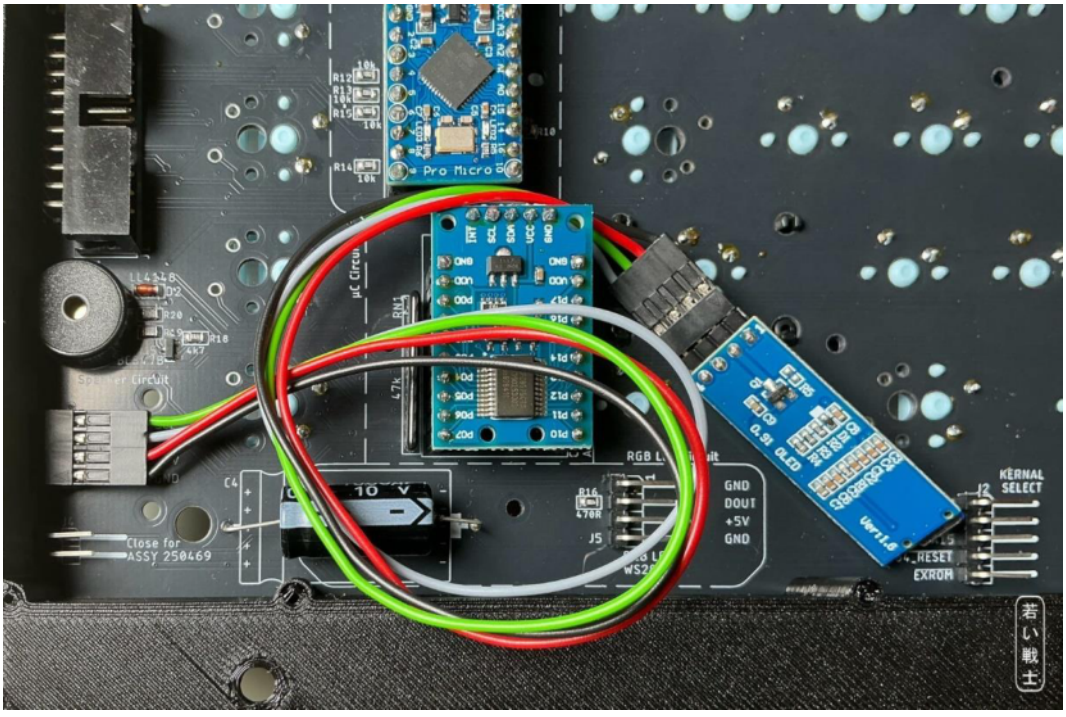


Figure 39: 0.91" OLED display connected to J7

The **5mm hole** in the PCB next to the left pin of the 1000µF capacitor is provided for **routing the display cable** through the board, if required.

6 Connector Pinouts

6.1 J1 - Keyboard Connector

2x10 box pin header 90°

Signal	Pin	Pin	Signal
GND	1	2	n.c.
RESTORE	3	4	+5V
ROW3	5	6	ROW6
ROW5	7	8	ROW4
ROW7	9	10	ROW2
ROW1	11	12	ROW0
COL0	13	14	COL6
COL5	15	16	COL4
COL3	17	18	COL2
COL1	19	20	COL7

6.2 J2 – Kernal Switcher

5 way pin header 90°

Pin	Kernal Switcher	Ultimate
1	A13	Left = uFreeze
2	A14	Menu = uMenu
3	A15	Right = uReset
4	RESET	Not relevant
5	EXROM	Not relevant

6.3 J3 & J8 – Power LED

3 way Molex KK2.54/KF2510, 90°

Pin	Signal
1	LED+
2	GND
3	LED+

6.4 J4 – Short Board Jumper

2 way pin header 90°

Place if short board

6.5 J5 – RGB LED Strip

4 way pin header 90°

Pin	Signal
1	GND
2	Din (of LED strip)
3	+5V
4	GND

6.6 J6 – Reserved

Do not place. Reserve pins of the pro micro.

5 way pin header

Pin	Signal
1	GND
2	RX
3	TX
4	A3
5	+5V

6.7 J7 – I²C Bus

4 way pin header, 90°

Pin	Signal
1	SCL
2	SDA
3	+5V
4	GND

7 Jumpers

7.1 JP1 – SW67/PREV to GND

JP1 connects **switch SW67 (pin 1)** to **GND**. This jumper is closed by default.

If no **microcontroller (M2)** is used, JP1 can be opened to allow **SW67** to be hardwired to any other switch in the keyboard matrix.

7.2 JP2 – SW68/NEXT to GND

JP2 connects **switch SW68 (pin 1)** to **GND**. This jumper is closed by default.

If no **microcontroller (M2)** is used, JP2 can be opened to allow **SW68** to be hardwired to any other switch in the keyboard matrix.

7.3 JP3 – Power LED

The **microcontroller (M2 = Pro Micro)** controls the power LED connected to **J3 or J8**.

If the Pro Micro is not installed and J3 or J8 is to be used for an **"always-on" power LED**, the corresponding jumper must be closed.

The advantage of this arrangement is that only the **keyboard cable** is required between the upper and lower halves of the case.

7.4 J4 – Short Board Jumper

See J4 – Short Board Jumper

8 Programming model

8.1 Pro Micro

Pro micro Pin	GPIO	Name	Direction	Description
1	D0/TX	TX	Tbd	Reserved (J6 pin 3)
2	D1/RX	RX	Tbd	Reserved (J6 pin 2)
5	SDA/D2	SDA	Bidir	I²C-Bus (M1 & J7 pin 2)
6	SCL/D3	SCL	Output	I²C-Bus (M1 & J7 pin 1)
7	A6/D4	/RESTORE	Input	RESTORE button (active low)
8	D5	SND_OUT	Output	Output for piezo buzzer
9	A7/D6	RGBOUT	Output	Output for RGB LED strip (J5 pin2)
10	D7	/INT6	Input	Interrupt from PCF8575 GPIO Module
11	A8/D8	/C64RES	Bidir*	Signal for resetting the C64 and shift lock status, active low
12	A9/D9	PWRLED	Output	Output for power led (J3&8), active high
13	D10/A10	/EXROM	Bidir*	EXROM signal, active low
14	D16	KSW_A15	Bidir*	Kernal address bit A15/ultimate freeze
15	D14	KSW_A14	Bidir*	Kernal address bit A14/ultimate menu
16	D15	KSW_A13	Bidir*	Kernal address bit A13/ultimate reset
17	A0	/SHORT_BRD	Input	J4 short board jumper, active low
18	A1	/P1	Input	SW67/PREV switch, active low
19	A2	/P2	Input	SW68/NEXT switch, active low
20	A3	A3	tbd	Reserved (J6 pin 4)

Bidir*: A pin marked with an asterisk is designed to emulate an **open-collector GPIO**. When the signal is low, the GPIO pin actively drives the output low. When the signal is "high," the GPIO pin is left floating (i.e. configured as an input). A **pull-up resistor** elsewhere in the circuit then pulls the signal to the required high level.

This is important when a digital signal can have **multiple sources**, such as **/C64RES**, which can originate from the C64 reset circuit, a reset button, or the Pro Micro.

It is particularly important for the **Ultimate button signals**, because the Ultimate 64/C64U uses **3.3 V logic levels**, which are not necessarily 5 V tolerant.

Note: GPIO stands for **General Purpose Input/Output**. A GPIO pin does not have a fixed function; its operation is determined by the circuit to which it is connected.

8.2 Keyboard Matrix/PCF8575

The **PCF8575 module (M1)** is a **16-bit GPIO expander** connected via the **I²C bus**.

Its primary purpose is to detect the keyboard matrix scan signals. If these signals are not detected, the Pro Micro uses the PCF8575 to **actively scan the keyboard matrix at 125 times per second**.

This provides the functionality required to use the keyboard as a **USB keyboard with a keyboard layout suitable for VICE (C64 emulator)**.

M1 GPIO	Signal	Discription
P00	ROW0	Keyboard return signals: These signals are **read** to detect the switches of the keyboard matrix, that are pressed.
P01	ROW1	
P02	ROW2	
P03	ROW3	
P04	ROW4	
P05	ROW5	
P06	ROW6	
P07	ROW7	
P10	COL0	Keyboard scan signal: A bit pattern is **written** to the columns of the keyboard matrix.
P11	COL1	
P12	COL2	
P13	COL3	
P14	COL4	
P15	COL5	
P16	COL6	
P17	COL7	

The **GPIO slave address** is selected using the solder bridges on the bottom side of the module (see Figure 20). The correct address is **0x20**.

9 Keycap Making

Options for obtaining or making the keycaps will be described or listed in the **/Keycaps/pdf/** subfolder.

At the time of writing, some options are still being evaluated.

10 3D Printables

A **mounting frame** is required to install the keyboard inside a **C64 or C64C**. It can be found in the **/3D printables/Mounting Frame/** subfolder.

An optional **stiffener** can be screwed to the solder side of the PCB. In my experience, it is not required, since the PCB already provides sufficient stability when screwed to the mounting frame.

A separate **plate** is not required. A plate is the level approximately **5 mm above the PCB**, with cutouts for the keyboard switches. It is normally used to align **3-pin switches**. Since **5-pin switches** are recommended for this keyboard, a plate is not necessary.

There are, however, plates that serve as the top cover of the **USB keyboard case**. These can be found in the **/3D printables/USB Keyboard Case/** subfolder.

There are two types of plate: **regular** and **classic**. The **classic** plate is only suitable if you want to use original Commodore keycaps from the 20th century. All other DIY and modern Commodore keycaps require the **regular** plate. The difference is the cutout and switch position for the **1.5U keycaps** (Shift, Control, Restore, F-keys, etc.).

Both the **mounting frame** and the **USB keyboard case** are designed to fit on a **25 cm-wide 3D printer bed**.

Figure 40: 3D printable mountinmg frame (top) and USB keyboard case (bottom)

11 Assembly

11.1 The Bill of Material

The complete circuit is divided into **subcircuits**, which can be populated optionally depending on the individual application.
The C64 keyboard PCB can always be **fully populated**, as none of the optional circuits interfere with the other functions, even if they are not used. The only disadvantage is that the keyboard will be **slightly more expensive than necessary** for a particular application.

The **BOM (Bill of Materials)** represents the fully populated version of the PCB. However, you may not require all components, depending on your intended use case.

The components are assigned to **functional groups**:

Short	Group
SL	Shift Lock Circuit
CTRL	Controller/pro micro
SPK	Piezo Buzzer
USBKB	USB-Keyboard
KSW	Kernal Switcher
RGB	RGB LED strip
I²C	I²C (OLED) Display

The **switches** and the **keyboard connector J1** are always required.

The **Shift Lock circuit** should normally be populated, unless you find Shift Lock annoying. If it is omitted, the **Shift Lock key has no function**.

Every other option requires the **controller (CTRL) / Pro Micro**.

The SPK (piezo buzzer) is recommended. It provides audible feedback for the status of a reset and for the additional PREV and NEXT switches.

If you do not intend to use the keyboard as a USB keyboard, you can leave out the USBKB group. Including these components does not cause any problems, however. For USB keyboard operation, a Pro Micro with a USB-C connector is highly recommended.

The Kernal switcher requires only a small number of additional components. In addition to everything marked KSW in the BOM, the two additional switches SW67 and SW68 (PREV and NEXT) are required to switch between the Kernals.

The RGB LED strip is optional, as are the components marked RGB. If these components are populated but the LED strip is not connected, they do not cause any problems. SW67 and SW68 are required to switch between the LED patterns. The default state is off—without the switches, there is no way to activate the lighting.

The I²C bus is intended for connecting an OLED display. It could also be used for other devices, such as IC temperature sensors, but this functionality is not implemented in the firmware yet. Since an OLED display requires either a transparent case or a suitable cutout, it is entirely optional.

11.2 Assembly of the PCB

11.2.1 SMD component

Not everybody likes SMD components, but in my opinion, they are required for this project. I solder the SMD components with solder paste and a hot plate, but neither is required. You do not need a fancy hot-air rework station, either.

The components used are 0805 passives (which are still a reasonable size), SOT-23 transistors, and an SO-14 IC. For most of my life, I have soldered components of this size with normal solder and a soldering iron. When soldering SMD components, the use of additional flux (gel or liquid) is highly recommended.

If you are concerned about the component size because your eyes are over 40 years old, consider getting a headband magnifier. It can make a significant difference when working with small components.

Keep in mind that small components can easily get lost. Order a few more than specified in the BOM. There are already plenty of chip resistors orbiting the Earth after jumping off my tweezers.

Another option is to order the PCBs with the SMD components already assembled. I have only used this service with JLCPCB so far. A pick-and-place file is included in the project data in the format preferred by JLCPCB, as is the BOM.

Since I have not yet ordered assembled PCBs, I cannot say how much additional preparation is required. Both files will need to be edited. First, the non-SMD components should be removed. It is unlikely that JLCPCB will not have the required components available. Once the component placement is displayed, check the orientation carefully. In particular, the diodes, transistors, and ICs may not be oriented correctly and will need to be checked and corrected.

If you decide to place the SMD components yourself, solder them first, before installing the larger components. A very fine soldering tip is not necessary. I prefer a beveled tip approximately 2 mm wide as my general-purpose tip and rarely use anything else. It is wider than the pin spacing of an SO-14 IC, but that is not a problem:

The flux will do the magic!

I have to admit that I used soldering tips that were far too fine for at least 40 years.

11.2.2 The Through Hole Components

11.2.2.1 The Pro Micro (M2)

The **Pro Micro** usually comes with two **12-pin headers** that are not soldered. The module is mounted using two corresponding **socket strips**, which are soldered to the keyboard PCB. This method makes it easier to install the module with properly aligned pins.

1. Plug the pin headers into the socket strips.
2. Insert the **socket strips—not the pin headers—**into the keyboard PCB.
3. Place the Pro Micro onto the pin headers, with the components facing upward.
4. Solder the pin headers to the Pro Micro.
5. Solder the socket strips to the keyboard PCB on the solder side. The USB connector must face the edge of the PCB, as indicated by the silkscreen.

11.2.2.2 The GPIO extension: PCF8575 Module

This module is, as stated previously, only required for the **USB keyboard version**.

The first step is to set the **I²C address bits** correctly. On the solder side of the module, there are three solder bridges labeled **A0, A1, and A2**. All three solder bridges must connect to **GND**; otherwise, the Pro Micro cannot communicate with the module. See Figure 20.

The module comes with two **10-pin headers**. It also includes a **5-pin 90° angled header**, which is not suitable for this project. Please use the **straight 5-pin header specified in the BOM** instead.

As with the Pro Micro, the module should be installed with the pins properly aligned. The installation procedure is the same as for the Pro Micro.

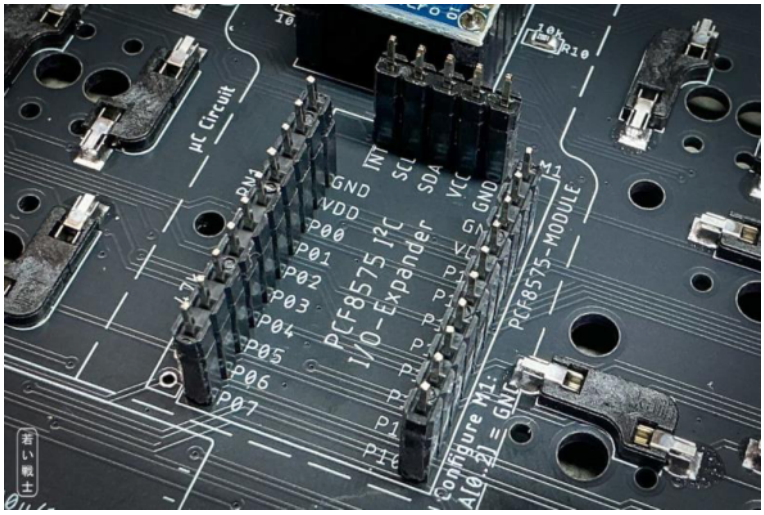


Figure 41: GPIO module - headers inserted (pin header up, socket stripe down

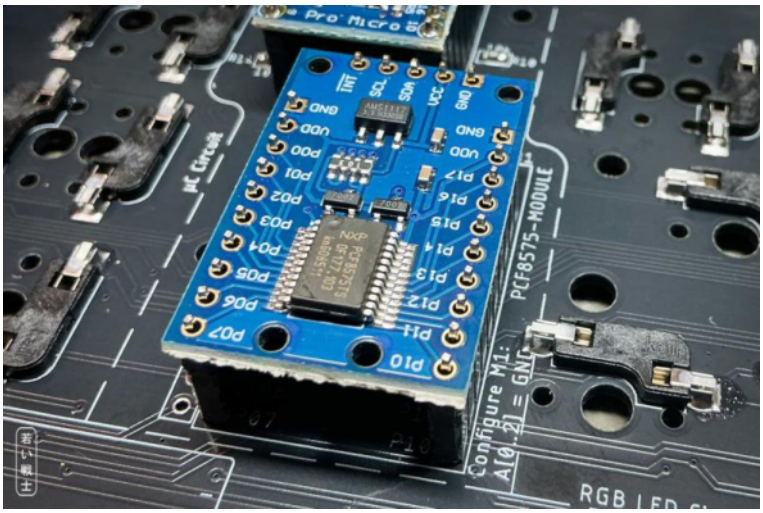


Figure 42: GPIO module before soldering

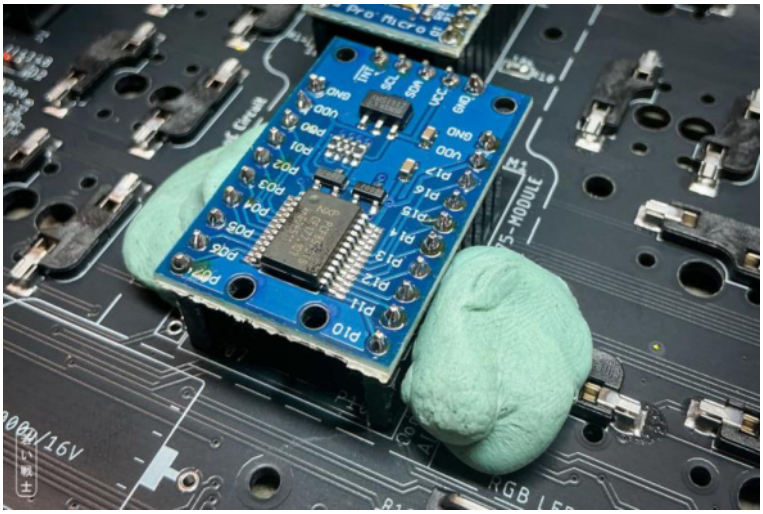


Figure 43: GPIO module after soldering (Held by blue tack)

For temporarily holding components in position, I use the popular **Blu Tack**. It is sufficiently sticky, can be reused many times, and is particularly useful for components that tend to fall out when the PCB is flipped over for soldering.

11.2.2.3 Remaining components

It does not matter whether the other **through-hole components** are soldered before or after the modules.

The **switches should be installed last**.

11.2.2.4 Stabilizers

It is a good idea to install the **stabilizers before soldering in the switches**. This leaves enough space to apply the moderate amount of force required to install them.

The wire of the **7U stabilizer** runs **above the spacebar switch**, while the wire of the **2U stabilizer** runs **below the RETURN switch** (see Figure 45).

Figure 44: 9U stabilizer

9U Stabilizer:
The **9U stabilizer** is only required if you want to use **classic keycaps**.

To make a 9U stabilizer, take another stabilizer and bend the replacement wire according to drawing **223-3-03-01** in the **/9U_stabilizer/** subfolder. The required material is **1.5 mm spring-steel wire** (see Figure 44). You will also find a documentation about making a 9U stabilizer wire in the said subfolder.

Note: The **C64U** and **Blingboard64** space bars use a **7U stabilizer**, which is relevant if you intend to use these keycaps.

Figure 45: Keyboard with stabilizers, switches already installed

11.2.2.5 Switches and SHIFT LOCK LED

Before soldering the keyboard switches, check whether the **LED should be installed before or after the Shift Lock switch**.

There are generally two types of switches:

- On some switches, the **LED is installed above the switch**.
- Switches **with** a **light guide** can have the LED installed underneath. In this case, the light guide must first be removed by pressing it out from below, and the **LED must be soldered before the switch is installed**.

The LED is **polarized**. The **shorter pin is the cathode (-)** and the **longer pin is the anode (+)**.

The high-brightness LED specified in the BOM can be somewhat confusing because its internal construction may make the polarity appear different. **Use the pin lengths to determine the polarity, not the internal appearance of the LED.**

The PCB silkscreen indicates the correct polarity. Unfortunately, on earlier PCB revisions this marking is covered by the switch once the switch has been installed. The polarity issue is corrected in later PCB revisions.

The **anode (+) points toward the RUN/STOP key**.

If your switches have light guides, **remove the light guide only from the SHIFT LOCK switch**. The light guides in all other switches should remain installed.

Figure 46: left to right: Switch with top entry LED configuration, with light guide, light guide removed

12 Cable Making

Please find the instructions on cable making in the /Cables/pdf/ sub-folder of this repository.

13 Wiring Up the Keyboard

13.1 USB Keyboard

The USB keyboard does not require any external wires.

13.2 Keyboard Matrix

The keyboard Matrix is wired to the C64 mainboard keyboard connector through the C64 Keyboard Connector PCB (project no. 224 in this repository). See Figure 12.

13.3 Kernal Switcher

The Kernal Switcher J2 is connected to the Kernal Adapter and the mainboard via the cable from drawing 233-3-02-00. See chapter 5.8, Figure 23, Figure 29 and Figure 30.

13.4 Ultimate Buttons

In Ultimate mode, the Kernal Switcher J2 can control the buttons pin header on the Ultimate/C64 Ultimate buttons pin header via the cable from drawing 233-3-03-00.

See chapter 5.9, Figure 31 and Figure 32.

13.5 OLED Display (optional)

The OLED displays have different pin outs. Just connect VCC to +5V, GND to GND, SDA to SDA, SCK to SCK. Use Dupont cables (self-fabricated or bought). See chapter 5.11.

13.6 The LED Strip cable (optional)

The LED strip cable is soldered to the WS2812B LED strip (see Figure 35 and Figure 36).

14 Series Production

If you plan to produce this project in series, the **hot-swap version** might be a good choice, as most of the components are SMD and can be machine-placed and soldered. Only the connectors and the sockets for the **Pro Micro** and **GPIO module** need to be soldered by hand. The standard version is ok for prefabrication of SMD places boards.

The **/JLCPCB Assembly/ folder** contains a BOM and a pick-and-place file (CP1) in a format compatible with the **JLCPCB PCBA (PCB assembly) process**. It might still require some work to get the assembly on track, though.

15 Product References and Disclaimer

Throughout this documentation, I mention specific **products**, **brand names**, and **suppliers**. These references are provided **solely for informational purposes** and are not intended as advertising or endorsements. I have not received any compensation or other benefits for mentioning them. I refer to these products and suppliers simply because I have used them myself and can therefore share my personal experience with them.

16 Revision History

16.1 Rev. 0

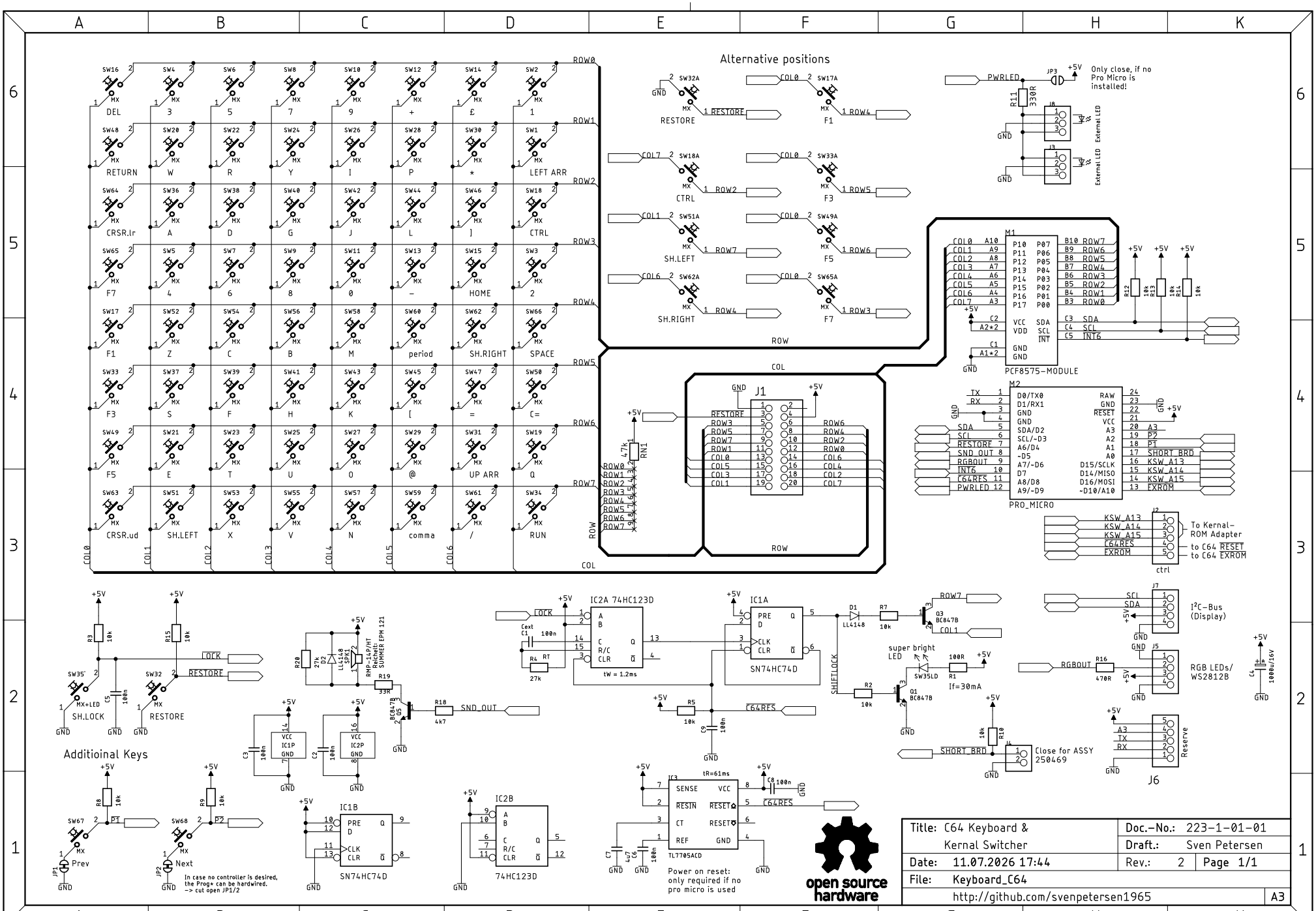
- First prototype: requires a botch wire for the shift lock circuit. Requires a Pro Micro controller.

16.2 Rev. 1

- Shift Lock Debounce circuit changed
- Power-on Reset added
- This requires a botch to suppress the power-on pulse. The problem is only effective, if no Pro Micro is populated

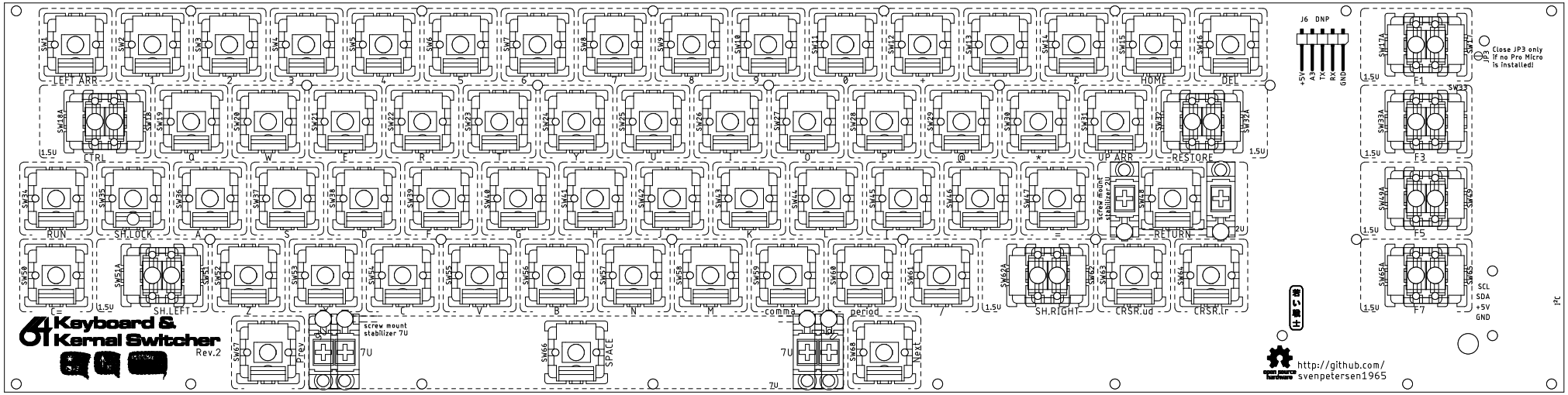
16.3 Rev. 2

- The Power-on Reset and Shift Lock circuit work properly now

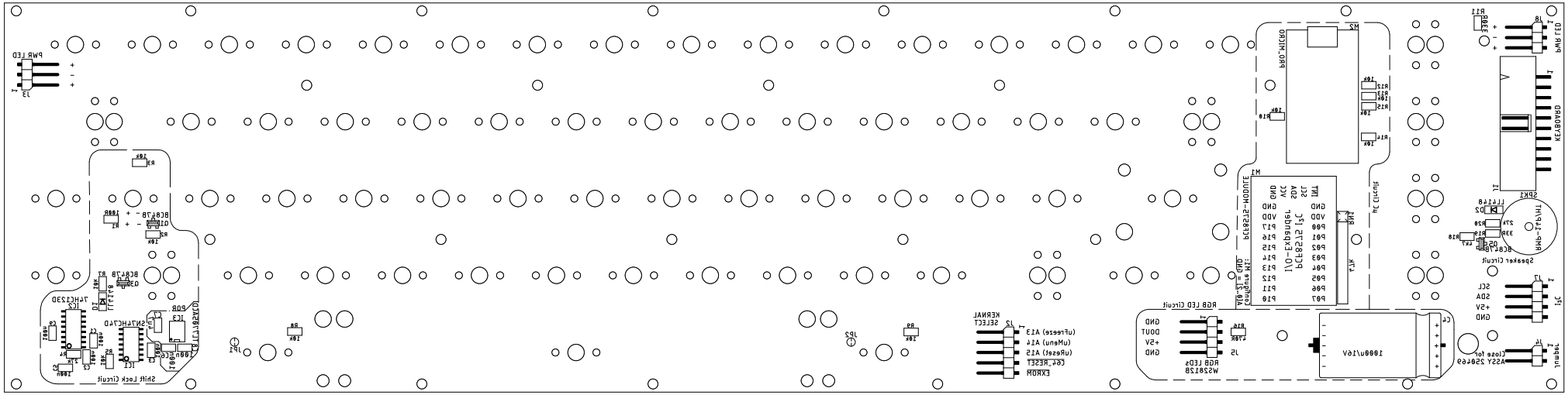


Title: C64 Keyboard & Kernal Switcher		Doc.-No.:	223-1-01-01
		Draft.:	Sven Petersen
Date:	11.07.2026 17:44	Rev.: 2	Page 1/1
File:	Keyboard_C64		
	http://github.com/svenpetersen1965		A3

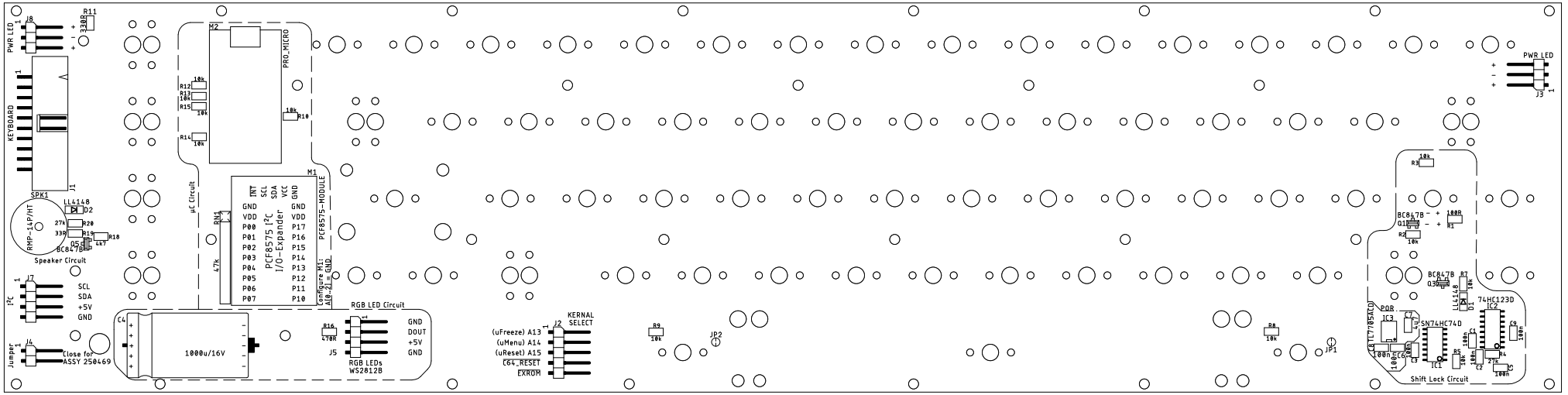
Sven Petersen 2026	Doc.-No.: 223-2-01-02	
	Cu: 35µ	Cu-Layers: 2
Keyboard_C64		
11.07.2026 17:45		Rev.: 2
placement component side		



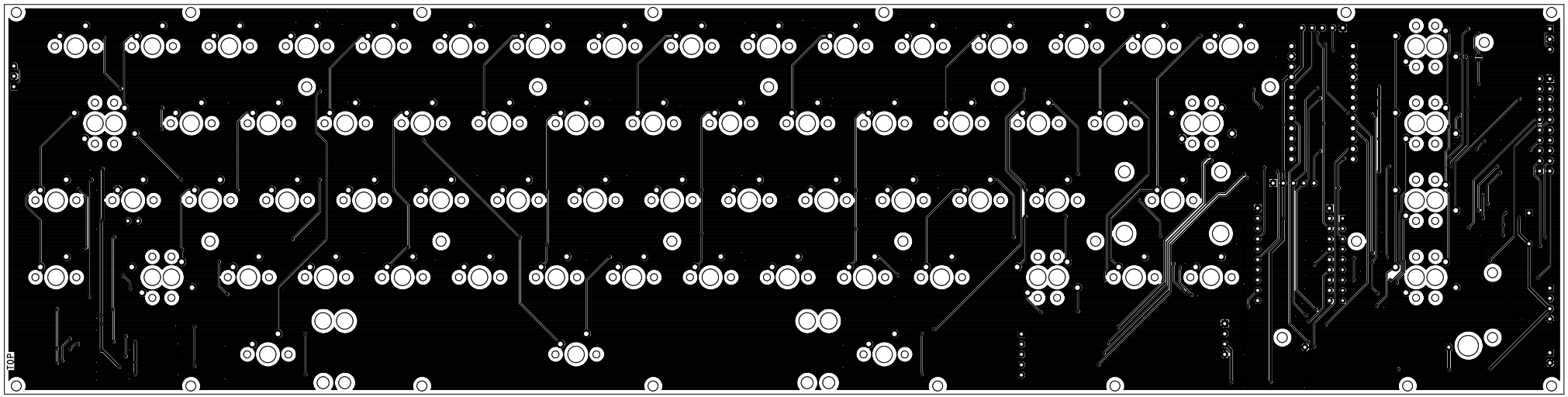
Sven Petersen 2026	Doc.-No.: 223-2-01-02	
	Cu: 35µ	Cu-Layers: 2
Keyboard_C64		
11.07.2026 17:45		Rev.: 2
placement solder side		



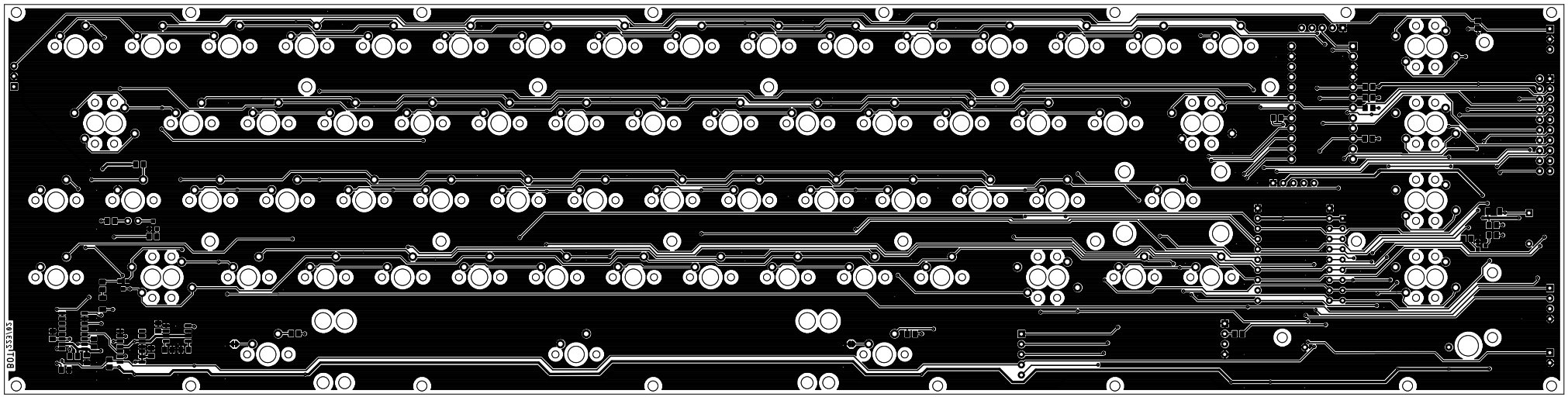
Sven Petersen 2026	Doc.-No.: 223-2-01-02	
	Cu: 35µ	Cu-Layers: 2
Keyboard_16k		
11.07.2026 17:45	Rev.: 2	
placement solder side		



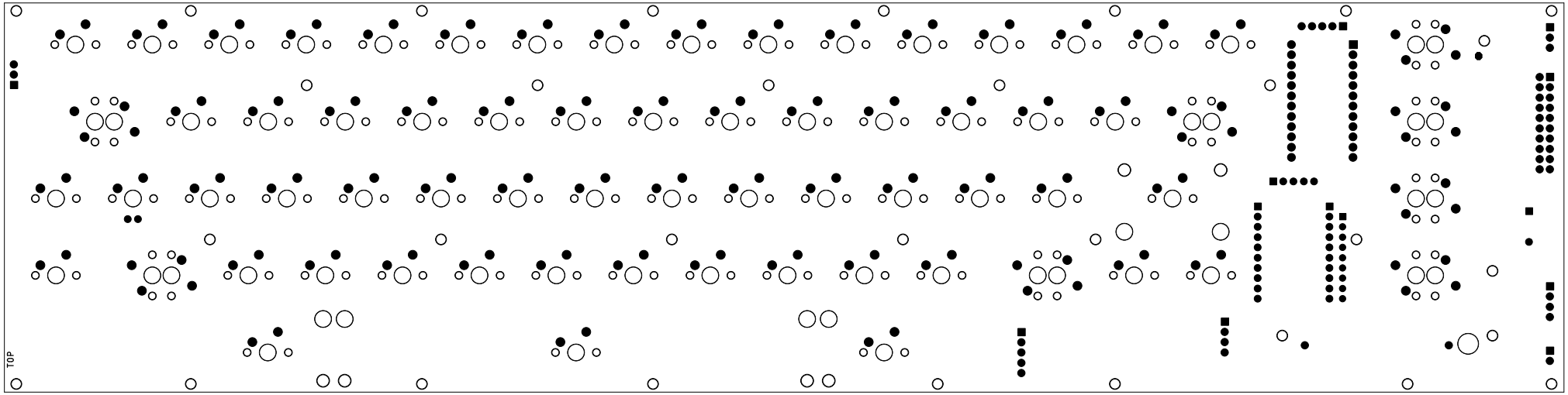
Sven Petersen 2026	Doc.-No.: 223-2-01-02	
	Cu: 35µ	Cu-Layers: 2
Keyboard_C64		
11.07.2026 17:45		Rev.: 2
top		



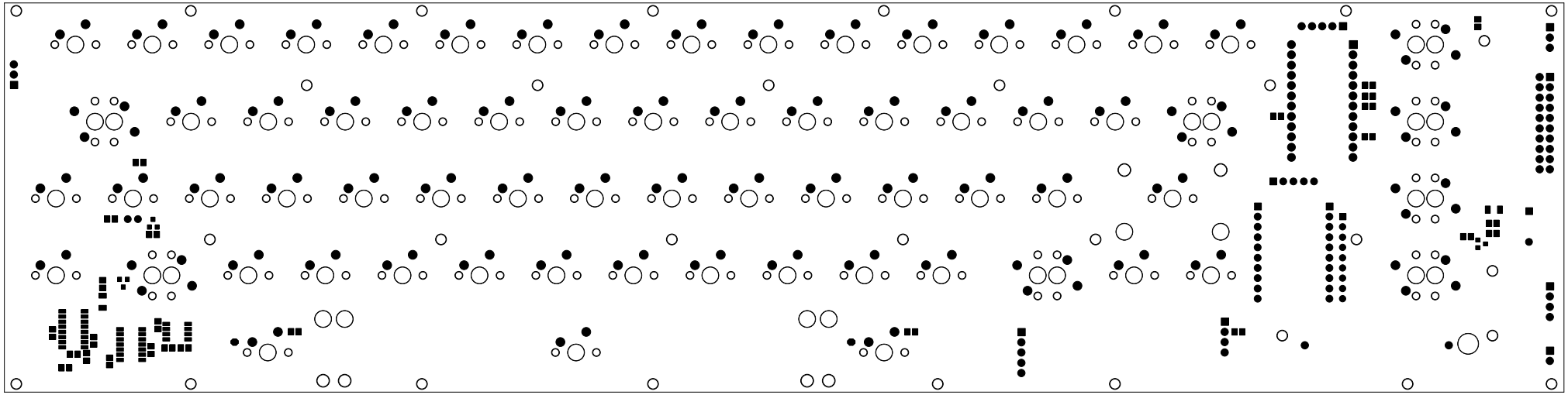
Sven Petersen 2026	Doc.-No.: 223-2-01-02	
	Cu: 35µ	Cu-Layers: 2
Keyboard_C64		
11.07.2026 17:45		Rev.: 2
bottom		



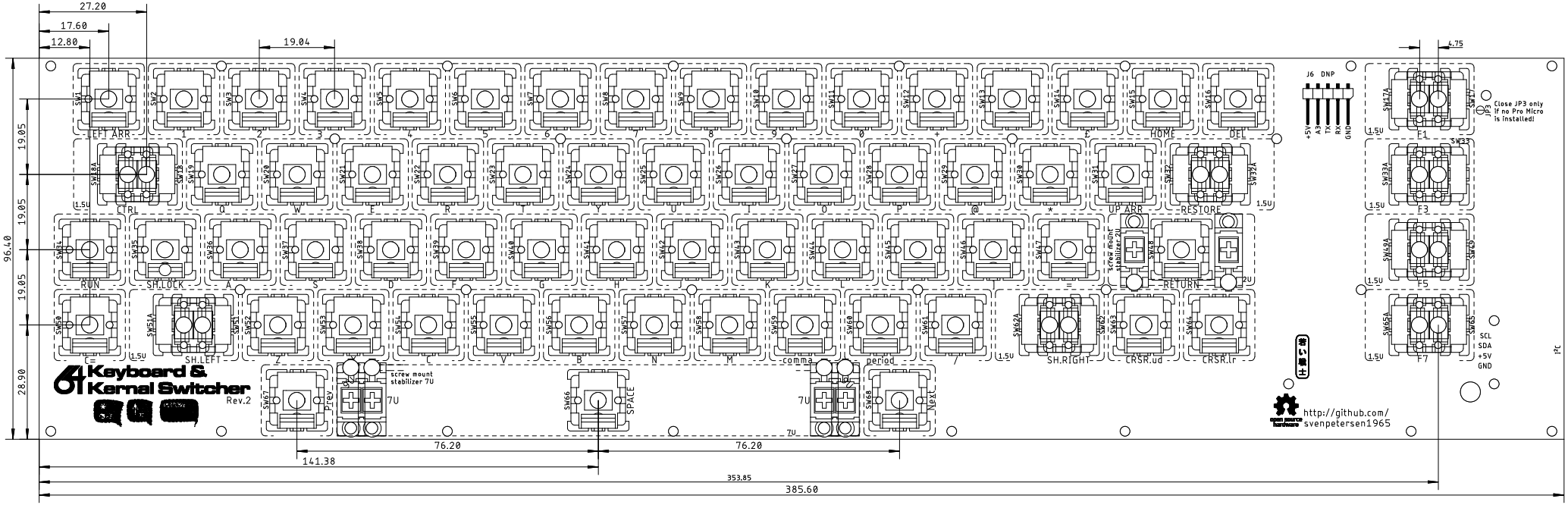
Sven Petersen 2026	Doc.-No.: 223-2-01-02	
	Cu: 35µ	Cu-Layers: 2
Keyboard_C64		
11.07.2026 17:45		Rev.: 2
stopmask component side		



Sven Petersen 2026	Doc.-No.: 223-2-01-02	
	Cu: 35µ	Cu-Layers: 2
Keyboard_C64		
11.07.2026 17:45		Rev.: 2
stopmask solder side		



Sven Petersen 2026	Doc.-No.: 223-2-01-02	
	Cu: 35µ	Cu-Layers: 2
Keyboard_C64		
11.07.2026 17:45		Rev.: 2
placement component side	measures	



C64 Mechanical Keyboard & Kernal Switcher

Standard Version Rev. 2

Functional Description

SW1 to SW66, with the exception of SW35, form the keyboard matrix of the C64 keyboard. It is identical to the original C64 keyboard matrix.

SW17A, SW18A, SW32A, SW33A, SW49A, SW51A, SW62A and SW65A are alternative positions for **1.5U keys**, which nowadays typically have a central stem. They are wired in parallel with their corresponding classic-position switches (without the "A").

SW32 is the RESTORE button. It is connected to the **/RESTORE** pin and GND, just like in the original keyboard. Unlike the original, it has a **10 kΩ pull-up resistor (R15)**, since the RESTORE signal is also required to have a defined state when the keyboard is not connected to a C64 mainboard.

SW35 is the SHIFT LOCK button. Unlike the original keyboard, where it is a latching switch and is electrically connected in parallel with the left SHIFT key, SW35 is connected to a dedicated **Shift Lock circuit**.

R3 is the pull-up resistor, and IC2A provides switch debouncing circuit.

IC3 and C7, C6 and C8 form the **power-on reset circuit** (POR) for the SHIFT LOCK flip-flop IC1A. **In case a Pro Micro (M2) is installed, the POR can stay unpopulated** since M2 will reset the SHIFT LOCK circuit.

IC1A toggles its output state every time the SHIFT LOCK button is pressed. R2 and Q1 drive the **SHIFT LOCK LED (SW35LD)**, while R1 is the current-limiting resistor.

The value is calculated as follows:

$$\mathrm{R1} = \frac{5\mathrm{V} - \mathrm{V_{F,SW35LD}}}{\mathrm{I_{F,SW35LD}}}$$

V_F is the forward voltage of the LED (approximately 2 V for a red LED; see the datasheet). I_F is the forward current, which should be below the maximum current specified in the LED datasheet. **100 Ω** is the minimum resistor value for an LED with a maximum forward current of 30 mA.

The diode **D1** blocks any reverse current from the base of **Q3**. Q3 is the **SHIFT LOCK switch**. This circuit is derived from the **SX-64 I/O board schematics**, which also does not use a latching SHIFT LOCK switch. It pulls **ROW7** low while **COL1** is pulled low by the C64 keyboard scan. This is electrically in parallel with the left SHIFT button, **SW51**.

J1 is a pin header that connects to the ribbon cable between the C64 keyboard and the C64 Keyboard Connector PCB.

M2 is the **Pro Micro microcontroller module**. It provides the advanced functions of the keyboard.

SW67 (PREV) and **SW68 (NEXT)** are the two additional keys that occupy the space to the left and right of the 7U space bar. They cannot be populated when a traditional **9U space bar** is used.

Q5 is the amplifier for the piezo buzzer. D2 is the freewheeling diode for the piezo buzzer, which can produce a voltage spike when subjected to mechanical impact.

J4 is the jumper that must be installed for a short-board C64. R10 is the pull-up resistor for the /SHORT_BRD signal.

J6 is a reserve connector with unused pins and supply voltage. TX and RX are the signals of the serial port, while A3 is an analog input.

J2 is the connector for the Kernal switcher, providing access to the upper address bits A13–A15 of the EPROM in the Kernal adapter. /C64RES is connected to the internal /RESET signal of a classic C64 mainboard. /EXROM is connected to the /EXROM signal of the classic C64 expansion port.

This is, how an EXROM Reset works:

With /EXROM = high (no cartridge connected), the CPU accesses the RAM instead. The 6510 CPU cannot distinguish between data read from RAM and data read from a cartridge ROM during the cartridge detection process. It therefore searches the first bytes of the cartridge memory area for the CBM80 signature.

Some programs deliberately leave a fake CBM80 signature in RAM. If the CPU finds this signature, it assumes that a cartridge is present and interprets the bytes preceding the signature as the cartridge's start address and NMI vector. These vectors can be manipulated by the program so that, after a RESET, the C64 does not return to the normal reset routine and the program memory cannot be read out normally.

By briefly asserting /EXROM during RESET, the CPU is forced to treat the memory area as cartridge ROM. The resulting cartridge detection process prevents the fake CBM80 signature left in RAM from being effective. This can circumvent a certain type of copy protection that manipulates the RESET and NMI (RESTORE) vectors.

J7 is the external I²C bus. It is intended for connecting a display. R12 and R13 are the pull-up resistors required by the Pro Micro's I²C bus.

J5 connects to the WS2812B RGB LED strip. R16 and C4 are components recommended in some LED strip application notes.

M1 is the GPIO module. It contains a TI PCF8575 I²C GPIO expander, which allows the Pro Micro to scan the keyboard matrix when USB keyboard functionality is required.

If the module is not used, other I²C devices or sensors could be connected to the bus. This would require additional software, however. One possible application would be connecting temperature sensors to monitor the temperatures of various ICs.

A power LED can be connected to either J3 or J8, which are located at opposite ends of the keyboard PCB. R11 is the current-limiting resistor and provides approximately 10 mA for a standard red or green LED.

PWRLED controls the connected LED. It is normally on and is switched off temporarily for signaling purposes. If M2 is not populated, closing JP3 provides a permanent supply for the LED, keeping it continuously on.

C64 Keyboard (Standard) Rev. 2
Bill of Material Rev. 2.0

Pos.	Qty	Value	Footprint	Ref.-No.	Comment	Group
1	1	223-2-01-02	2 Layer	_PCB Rev. 2	2 layer, Cu 35µ, HASL, 385.6mm x 96.4mm, 1.6mm FR4	
2	4	100n	0805	C1, C2, C3, C9	Capacitor SMD 0805, 25V, X7R, ±10% or better	SL
3	1	1000u/10V	C30X16L	C4	electrolytic capacitor, axial leads, maximum length 30mm, maximum ∅ 16mm. E.g. Reichelt AX 105 1000/10	RGB
4	1	100n	0805	C5	Capacitor SMD 0805, 25V, X7R, ±10% or better	SL
5	2	100n	0805	C6, C8	Capacitor SMD 0805, 25V, X7R, ±10% or better	POR
6	1	4u7	0805	C7	Capacitor SMD 0805, 25V, X7R, ±10% or better	POR
7	1	LL4148	SOD80C	D1	Diode, SMD	SL
8	1	LL4148	SOD80C	D2	Diode, SMD	SPK
9	1	SN74HC74D	SO-14	IC1	SMD IC, e.G. TI	SL
10	1	74HC123D	SO-16	IC2	SMD IC, e.G. TI	SL
11	1	TL7705ACD	SO8	IC3	SMD IC, e.G. TI	POR
12	1	box pin header, 2x10, 90°	2X10WH	J1	standard 2x10 pin box pin header, e.g. Reichelt.de WSL 20W	
13	1	pin header, 5p/90°, 2.54mm	1X05/90	J2	standard 90° pin header, e.g. Reichelt BKL 10120522	KSW
14	1	pin header, 5p/90°, 2.54mm	1X05/90	J6	DO NOT PLACE	
15	2	KF2510 , 3p/90°, 2.54mm	1X03/90	J3, J8	KF2510 (AliExpress) or Molex KK 2.54 (3p, 90°)	
16	1	pin header, 2p/90°, 2.54mm	1X02/90	J4	standard 90° pin header, e.g. Reichelt BKL 10120522	KSW

C64 Keyboard (Standard) Rev. 2
Bill of Material Rev. 2.0

Pos.	Qty	Value	Footprint	Ref.-No.	Comment	Group
17	1	Jumper, 2.54mm		(J4)	standard 2.54mm jumper, only requird for short boards	KSW
18	1	pin header, 4p/90°, 2.54mm	1X04/90	J5	standard 90° pin header, e.g. Reichelt W+P 946-144	RGB
19	1	pin header, 4p/90°, 2.54mm	1X04/90	J7	standard 90° pin header, e.g. Reichelt W+P 946-144	I²C
20	1	PCF8575-MODULE	PCF8575MODUL	M1	Ali Express PCF8575 module. Read module description. Configuration of solder bridges requires	USBKB
21	2	10 way socket for pin header 2.54mm		(M1)	precision sockets, 2.54mm, 1x10, straight. E.g. Reichelt MPE 115-1-010	USBKB
22	1	5 way socket for pin header 2.54mm		(M1)	precision sockets, 2.54mm, 1x5, straight. E.g. Reichelt MPE 115-1-005	USBKB
23	1	5 way pin header, 2.54mm		(M1)	straight, e.g. Reichelt BKL 10120503	USBKB
24	1	PRO_MICRO	PRO_MICRO	M2	Ali Express pro micro module. USB-C connector prefered. Read module description	CTRL
25	2	12 way socket for pin header 2.54mm		(M2)	precision sockets, 2.54mm, 1x12, straight. E.g. Reichelt MPE 115-1-012	CTRL
26	3	BC847B	SOT23	Q1, Q3, Q5		
27	1	100R*	0805	R1	Resistor SMD 0805, 5% or better. Read Module Description	SL
28	1	330R	0805	R11	Resistor SMD 0805, 5% or better	
29	1	470R	0805	R16	Resistor SMD 0805, 5% or better	RGB
30	1	4k7	0805	R18	Resistor SMD 0805, 5% or better	SPK
31	1	33R	0805	R19	Resistor SMD 0805, 5% or better	SPK
32	11	10k	0805	R2, R3, R5, R7	Resistor SMD 0805, 5% or better	SL
33	11	10k	0805	R8, R9, R10, R12, R13, R14, R15	Resistor SMD 0805, 5% or better	

C64 Keyboard (Standard) Rev. 2
Bill of Material Rev. 2.0

Pos.	Qty	Value	Footprint	Ref.-No.	Comment	Group
34	1	27k	0805	R20	Resistor SMD 0805, 5% or better	SPK
35	1	27k	0805	R4	Resistor SMD 0805, 5% or better	SL
36	1	47k	RN-9	RN1	Resistor network, 8 resistors/9 pins. E.g. Reichelt SIL 9-8 47K	USBKB
37	1	RMP-14P/HT	RMP-14P/HT	SPK1	piezzo buzzer, ∅14mm, pin distance 7.6mm, EKULIT (Reichelt SUMMER EPM 121)	SPK
38	58	Cherry MX compatible switch, 5 Pin	CHERRYMX	SW1, SW2, SW3, SW4, SW5, SW6, SW7, SW8, SW9, SW10, SW11, SW12, SW13, SW14, SW15, SW16, SW19, SW20, SW21, SW22, SW23, SW24, SW25, SW26, SW27, SW28, SW29, SW30, SW31, SW34, SW35, SW36, SW37, SW38, SW39, SW40, SW41, SW42, SW43, SW44, SW45, SW46, SW47, SW48, SW50, SW52, SW53, SW54, SW55, SW56, SW57, SW58, SW59, SW60, SW61, SW63, SW64, SW66	Cherry MX compatible switches of your choice. Read the module description.	
39	8	Cherry MX compatible switch, 5 Pin	CHERRYMX	SW17 or SW17A, SW18 or SW18A, SW32 or SW32A, SW33 or SW33A, SW49 or SW49A, SW51 or SW51A, SW62 or SW62A, SW65 or SW65A	SW** = for original keycaps, SW**A = for modern keycaps. Read module description	
40	2	Cherry MX compatible switch, 5 Pin	CHERRYMX	SW67, SW68		KSW and RGB

C64 Keyboard (Standard) Rev. 2
Bill of Material Rev. 2.0

Pos.	Qty	Value	Footprint	Ref.-No.	Comment	Group
41	1	LED (high brightness)	LED 3mm	(SW35)	e.G. Kingbright L-7104SRC-J4 (Reichelt LED 3-4500 RT)	SL
42	1	Stabilizer, screw in type 7U		(SW66)	(for 7U space bar or 9U spacebar of C64U/Blingboard64 (CBMStuff.com). Read module description)	
43	1	Stabilizer, screw in type 2U		(SW48)	for RETURN key	
44	58	1U, blank keycaps			for MX compatibel switches, DSA profile. Read module description	
45	8	1.5U, blank keycaps			for MX compatibel switches, DSA profile. Read module description	
46	1	2U, blank keycap		(SW48)	for MX compatibel switches, DSA profile. Read module description	
47	1	7U space bar		(SW66)	for MX compatibel switches, DSA profile. Read module description	
48	25	C 2.2 x 6,5 H			screw, self tapping, DIN 7981	
49	2	C 2.9 x 9,5 H			screw, self tapping, DIN 7981	
50	2	3.2mm washer				
51	1	3D printed keyboard frame			Rev. 1	
52	1	ribbon cable + connector PCB			Project No. 224	
53	1	kernal switcher cable			Drawing 223-3-001-00. Read module description	KSW
54	1	optional WS2812B RGB LED strip			Read module description	RGB
55	1	optional OLED display			Read module description	I²C

C64 Keyboard (Standard) Rev. 2
Bill of Material Rev. 2.0

Pos.	Qty	Value	Footprint	Ref.-No.	Comment	Group
Grp.	SL	Shift Lock Circuit				
	POR	Power on reset				
	RGB	RGB LED strip				
	SPK	Piezo Buzzer				
	KSW	Kernal Switcher				
	USBKB	USB Keyboard				
	I²C	I²C (OLED) Display				
	CTRL	Controller/pro micro				